

UNIVERSITATEA „LUCIAN BLAGA” DIN SIBIU
FACULTATEA DE ȘTIINȚE

Programul de masterat
Sisteme și Tehnologii Informatice Avansate

LUCRARE DE DISERTAȚIE

Coordonator științific

Șef lucr. univ. dr. Mircea-Adrian Mușan

Absolvent

Cormoș Sandra-Andreea

Sibiu
2026

UNIVERSITATEA „LUCIAN BLAGA” DIN SIBIU
FACULTATEA DE ȘTIINȚE

Programul de masterat:
Sisteme și Tehnologii Informatice Avansate

**Platforma web interactiva care
intergreaza elemente de AI pentru
generarea de continut dinamic**

Coordonator științific
Șef lucr. univ. dr. Mircea-Adrian Mușan

Absolvent
Cormoș Sandra-Andreea



Sibiu
2026

Abstract

Lucrarea prezintă proiectarea și implementarea aplicației LearnAI, o platformă web gamificată care utilizează inteligența artificială generativă pentru crearea dinamică a întrebărilor. Soluția urmărește diversificarea conținutului, configurarea sesiunilor după categorie și dificultate și păstrarea progresului utilizatorului.

Aplicația utilizează React și TypeScript pentru interfață, ASP.NET Core pentru backend, Firebase pentru autentificare și persistența datelor și modelul GPT-4o mini prin OpenAI Chat Completions API. Backend-ul solicită răspunsuri conforme unei scheme JSON stricte și verifică existența întrebării, numărul variantelor și unicitatea răspunsului corect. Platforma include teste personalizate, progres prin XP și niveluri, streak-uri, clasament și avataruri configurabile.

Validarea tehnică a inclus testele automate ale parserului și validatorului, build-ul frontend și verificări ale endpoint-urilor principale. Rezultatele confirmă funcționarea componentelor testate, fără a demonstra automat corectitudinea factuală a tuturor întrebărilor sau eficiența pedagogică. Lucrarea delimitează aceste aspecte și propune verificarea prin surse, evaluarea unui eșantion de întrebări și testarea cu utilizatori.

Cuvinte-cheie: inteligență artificială generativă, educație digitală, gamificare, React, ASP.NET Core, Firebase, OpenAI.

Cuprins

INTRODUCERE	1
CAPITOLUL 1. STADIUL ACTUAL AL CERCETĂRII.....	2
1.1. PLATFORME DE E-LEARNING ȘI EDUCAȚIA DIGITALĂ	2
1.2. CONCEPTE GENERALE PRIVIND GENERAREA PROCEDURALĂ DE CONȚINUT	3
1.2.1. Controlul utilizatorului asupra conținutului.....	3
1.3. FUNDAMENTELE INTELIGENȚEI ARTIFICIALE	4
1.3.1. AI, machine learning și deep learning: diferențe	4
1.3.2. Istoria inteligenței artificiale.....	5
1.3.3. Aplicații ale inteligenței artificiale	6
1.3.4. Utilizarea inteligenței artificiale în educație și generarea conținutului.....	7
1.4. DOVEZI PRIVIND UTILIZAREA AI ȘI A GAMIFICĂRII ÎN EDUCAȚIE	8
1.5. ANALIZA UNOR SOLUȚII EXISTENTE.....	9
1.6. MOTIVAȚIA ȘI CONTRIBUȚIA PROPRIE	9
CAPITOLUL 2. ANALIZA PROBLEMEI ȘI CERINȚELE SISTEMULUI.....	11
2.1. PROBLEMA DIVERSITĂȚII CONȚINUTULUI ÎN E-LEARNING	11
2.2. COSTURILE DE MENTENANȚĂ.....	11
2.3. COSTURILE DEZVOLTĂRII CONȚINUTULUI	12
2.4. DIFICULTATEA MENȚINERII IMPLICĂRII ȘI MOTIVAȚIEI UTILIZATORILOR.....	12
2.5. CERINȚE FUNCȚIONALE.....	12
2.6. CERINȚE NEFUNCȚIONALE	13
2.7. ACTORI ȘI FLUX PRINCIPAL	13
CAPITOLUL 3. PROIECTAREA ȘI ARHITECTURA SOLUȚIEI.....	14
3.1. STRATEGII PENTRU GENERAREA ȘI DIVERSIFICAREA CONȚINUTULUI EDUCAȚIONAL.....	14
3.1.1. Generarea dinamică a întrebărilor.....	14
3.1.2. Categori definite de utilizator	14
3.1.3. Beneficiile utilizării modelului GPT-4o mini	15
3.2. SOLUȚII PENTRU REDUCEREA COSTURILOR DE MENTENANȚĂ ȘI DEZVOLTARE.....	16
3.2.1. Avantajele Integrării Firebase.....	16
3.2.2. Comparatie cu alte soluții	17
3.3. GAMIFICAREA PROCESULUI DE ÎNVĂȚARE	17
3.3.1. Beneficiile Gamificării în Educație	18
3.3.2. Niveluri, XP și Streak-uri, Clasamente și Sisteme de Recompensare	18

CAPITOLUL 4. IMPLEMENTAREA ȘI DESCRIEREA APLICAȚIEI	20
4.1. ARHITECTURA GENERALĂ A SISTEMULUI.....	20
4.2. DESIGNUL INTERFEȚEI UTILIZATOR.....	21
4.2.1. Pagina de Autentificare	21
4.2.2. Dashboard-ul Utilizatorului	23
4.2.3. Configurarea Testelor	25
4.2.4. Ecranul de Rezolvare a Testelor	26
4.2.5. Setări și profil utilizator	28
4.3. IMPLEMENTARE TEHNICĂ	31
4.3.1. Implementarea frontend-ului cu React și TypeScript	31
4.3.2. Implementarea backend-ului cu ASP.NET Core	33
4.3.3. Integrarea Firebase	35
4.3.4. Integrarea OpenAI	38
4.3.5. Implementarea testelor personalizate.....	40
4.3.6. Implementarea mecanismelor de gamificare	43
4.3.7. Securitatea și validarea datelor	45
CAPITOLUL 5. TESTARE ȘI EVALUARE	48
5.1. OBIECTIVELE TESTĂRII.....	48
5.2. TESTE AUTOMATE BACKEND.....	48
5.3. VERIFICAREA FRONTEND-ULUI ȘI A API-ULUI	48
5.4. EVALUAREA CANTITATIVĂ A MODELULUI GENERATIV	48
5.5. LIMITĂRILE EVALUĂRII	49
CAPITOLUL 6. ÎMBUNĂTĂȚIRI VIITOARE	50
6.1. VERIFICAREA FACTUALĂ A ÎNTREBĂRIILOR GENERATE.....	50
6.2. GENERAREA TESTELOR DIN DOCUMENTE	50
6.3. PERSONALIZAREA ADAPTIVĂ A DIFICULTĂȚII.....	50
6.4. SECURITATE ȘI CONTROLUL UTILIZĂRII API-ULUI.....	51
6.5. OPTIMIZAREA PERFORMANȚEI ȘI A COSTURILOR	51
6.6. EXTINDEREA FORMATELOR DE ÎNTREBĂRI	51
6.7. MOD MULTIPLAYER.....	52
6.8. TESTARE ȘI EVALUARE EXTINSĂ	52
CAPITOLUL 7. CONCLUZII.....	53
BIBLIOGRAFIE	55

INTRODUCERE

Rezultatul acestui proiect are scopul de a demonstra cunoștințele și competențele dobândite pe parcursul studiilor de master în domeniul informaticii, prin proiectarea, dezvoltarea și implementarea unei platforme educaționale moderne asistate de inteligență artificială. Dezvoltarea aplicației a implicat parcurgerea tuturor etapelor specifice unui proiect software, de la analiză și proiectare până la implementare, testare și evaluare, utilizând tehnologii și practici actuale din industrie.

LearnAI este o platformă web gamificată care utilizează inteligența artificială pentru generarea succesivă a întrebărilor de trivia. Utilizatorul selectează categoria și dificultatea, iar întrebările anterioare sunt folosite pentru reducerea repetării în aceeași sesiune. Aplicația nu adaptează încă automat dificultatea pe baza performanței; această funcționalitate este propusă ca dezvoltare viitoare.

Unul dintre obiectivele principale ale platformei este reducerea caracterului repetitiv întâlnit frecvent în aplicațiile educaționale tradiționale. Prin utilizarea generării automate de conținut și a mecanismelor de context oferite de modelele AI, LearnAI urmărește să ofere întrebări variate și să minimizeze repetarea acestora, contribuind la menținerea interesului și implicării utilizatorilor pe termen lung.

Pe lângă funcționalitățile bazate pe inteligență artificială, platforma integrează elemente de gamificare precum niveluri, puncte de experiență, streak-uri și clasamente, menite să stimuleze motivația și participarea activă a utilizatorilor. Aceste mecanisme transformă procesul de învățare într-o experiență mai atractivă și mai captivantă.

Prin funcționalitățile sale, LearnAI își propune să demonstreze modul în care inteligența artificială poate fi integrată într-un sistem educațional modern pentru a îmbunătăți generarea de conținut, personalizarea procesului de învățare și implicarea utilizatorilor.

În realizarea acestei lucrări a fost utilizat OpenAI GPT pentru îmbunătățirea clarității exprimării și pentru asistență în procesul de redactare. Conținutul a fost verificat, revizuit și adaptat de autor, care își asumă responsabilitatea pentru forma finală a lucrării.

CAPITOLUL 1. Stadiul actual al cercetării

1.1. Platforme de e-learning și educația digitală

Transformarea digitală a societății a influențat semnificativ și domeniul educației, conducând la apariția și dezvoltarea platformelor de e-learning. Acestea reprezintă sisteme software care permit furnizarea, gestionarea și accesarea conținutului educațional prin intermediul internetului, oferind utilizatorilor posibilitatea de a învăța independent de locație și de intervalul orar disponibil.

În ultimii ani, interesul pentru educația digitală a crescut considerabil atât în mediul academic, cât și în cel profesional. Dezvoltarea tehnologiilor web, accesul extins la dispozitive conectate la internet și nevoia de învățare continuă au contribuit la adoptarea pe scară largă a platformelor educaționale online. Acestea sunt utilizate pentru cursuri universitare, programe de formare profesională, certificări și dezvoltarea competențelor necesare pe piața muncii.

Platformele de e-learning oferă numeroase avantaje comparativ cu metodele tradiționale de învățare. Utilizatorii pot accesa conținutul educațional în propriul ritm, pot relua materialele de studiu ori de câte ori este necesar și beneficiază de o flexibilitate ridicată în organizarea procesului de învățare. De asemenea, integrarea elementelor multimedia, precum imagini, videoclipuri, animații și exerciții interactive, contribuie la creșterea gradului de implicare și la îmbunătățirea experienței educaționale.

Cu toate acestea, platformele educaționale tradiționale se confruntă cu o serie de provocări. În multe cazuri, conținutul educațional este static și necesită actualizări manuale, ceea ce poate conduce la costuri ridicate de mentenanță și la dificultăți în menținerea materialelor actualizate. În plus, utilizatorii pot întâmpina aceleași întrebări și exerciții în mod repetat, ceea ce reduce gradul de implicare și eficiența procesului de învățare.

Pentru a răspunde acestor provocări, platformele moderne încep să integreze tehnologii bazate pe inteligență artificială, capabile să personalizeze experiența utilizatorului și să genereze conținut educațional adaptat nevoilor individuale. În paralel, elementele de gamificare sunt utilizate pentru a crește motivația și implicarea utilizatorilor. Aceste direcții reprezintă fundamentul pe care a fost dezvoltată platforma LearnAI, prezentată în cadrul acestei lucrări.

Una dintre provocările identificate în cadrul platformelor educaționale moderne este necesitatea furnizării unui volum mare de conținut diversificat și relevant pentru utilizatori. Crearea manuală a acestui conținut poate implica costuri ridicate și un efort considerabil de dezvoltare și mentenanță. În acest context, generarea automată a conținutului reprezintă o soluție promițătoare pentru reducerea acestor limitări și pentru oferirea unor experiențe de învățare mai personalizate. Unul dintre conceptele care stau la baza acestei abordări este generarea procedurală de conținut, prezentată în secțiunea următoare.

1.2. Concepte generale privind generarea procedurală de conținut

Generarea procedurală de conținut (Procedural Content Generation - PCG) are numeroase utilizări în domeniul aplicațiilor software [1]. Una dintre cele mai importante este creșterea varietății conținutului oferit utilizatorilor, deoarece conținutul generat dinamic poate conduce la experiențe diferite de utilizare. De asemenea, PCG a fost utilizată pentru a depăși anumite limitări tehnice și pentru a crea sisteme care se adaptează nivelului de cunoștințe, abilităților sau preferințelor utilizatorului, fie în timpul utilizării aplicației, fie offline.

Unul dintre primele exemple de utilizare a generării procedurale de conținut este jocul „Rogue”, lansat în anul 1980 [1].

„Rogue” este un joc de tip dungeon crawler creat în anii 1980. Jucătorii explorează nivelurile unei temnițe pentru a găsi Amuleta lui Yendor. Pe parcurs, aceștia întâlnesc monștri și colectează diverse comori, precum arme și poțiuni. Jocul este bazat pe ture și utilizează grafică ASCII. Nivelurile și obiectele din joc sunt generate procedural, oferind o experiență unică la fiecare sesiune și demonstrând potențialul generării procedurale de conținut în crearea unor experiențe variate pentru utilizatori.

Un concept strâns legat de varietatea conținutului este adaptabilitatea, care reprezintă utilizarea generării procedurale de conținut pentru ajustarea conținutului în funcție de acțiunile, preferințele sau nivelul de cunoștințe al utilizatorului. Adaptabilitatea reprezintă o modalitate de a personaliza experiența oferită de o aplicație și de a răspunde nevoilor unui grup divers de utilizatori, fiecare având obiective, stiluri de învățare și niveluri de experiență diferite.

În contextul platformelor educaționale moderne, generarea procedurală de conținut poate fi utilizată pentru crearea dinamică a întrebărilor, testelor și exercițiilor, contribuind la reducerea repetitivității și la oferirea unei experiențe de învățare personalizate pentru fiecare utilizator.

1.2.1. Controlul utilizatorului asupra conținutului

Multe aplicații care utilizează generarea procedurală de conținut nu permit utilizatorului acces direct la mecanismul de generare. În cazurile în care acest acces este oferit, există două forme principale de control: indirect și direct.

Controlul indirect este frecvent utilizat în sistemele care se adaptează în timp real comportamentului utilizatorului. Un exemplu este proiectul „Polymorph” [2], unde dificultatea conținutului este ajustată automat în funcție de performanța utilizatorului. Astfel, anumite acțiuni pot determina sistemul să genereze conținut mai accesibil sau mai complex. În mod similar, platformele educaționale moderne pot adapta dificultatea întrebărilor în funcție de rezultatele obținute de utilizator, oferind o experiență de învățare personalizată.

Pe de altă parte, controlul direct oferă utilizatorului acces explicit la anumite mecanisme de generare a conținutului, permițându-i să influențeze rezultatul final. Un exemplu este jocul „The Sims” [3], unde utilizatorii pot personaliza aproape toate aspectele personajelor create, de la aspectul fizic până la trăsăturile de personalitate. Acest nivel ridicat de control încurajează

creativitatea și permite obținerea unor experiențe unice. Într-o platformă educațională, un exemplu similar de control direct îl reprezintă posibilitatea utilizatorului de a selecta domeniul de studiu, nivelul de dificultate sau tipul întrebărilor generate.

Există și aplicații precum „Minecraft” [1], unde utilizatorul nu are acces direct la algoritmul de generare, dar poate influența anumiți parametri ai conținutului generat. De exemplu, acesta poate configura dimensiunea lumii, tipurile de biome sau alte caracteristici generale, fără a controla efectiv conținutul care va fi generat. Fiecare lume este generată pornind de la un seed, adică o valoare numerică utilizată pentru inițializarea algoritmului de generare. Utilizarea aceluiași seed produce întotdeauna aceeași lume, permițând recrearea exactă a conținutului generat. Deoarece seed-urile sunt stocate pe 64 de biți, există aproximativ 18,4 quintilioane de valori posibile și, implicit, un număr extrem de mare de lumi unice care pot fi generate.

În LearnAI, utilizatorul controlează direct categoria și dificultatea întrebărilor. Sistemul folosește istoricul întrebărilor din sesiunea curentă pentru reducerea repetării, dar nu adaptează încă automat dificultatea după performanță. O astfel de personalizare este analizată ca direcție de dezvoltare viitoare.

1.3. Fundamentele inteligenței artificiale

În lumea aflată într-o continuă evoluție, inteligența artificială (AI) nu mai reprezintă doar un instrument tehnologic, ci o forță transformatoare care influențează modul în care trăim, gândim și învățăm. Așa cum este subliniat în următorul citat: „Artificial intelligence (AI) systems already greatly impact our lives — they increasingly shape what we see, believe, and do. Based on the steady advances in AI technology and the significant recent increases in investment, we should expect AI technology to become even more powerful and impactful in the following years and decades” [4].

În acest capitol al lucrării va fi explorat potențialul inteligenței artificiale, cu accent pe aplicațiile sale în domeniul educațional și în generarea automată de conținut. Odată cu dezvoltarea modelelor moderne de inteligență artificială, au apărut noi oportunități pentru personalizarea experienței de învățare, automatizarea procesului de creare a conținutului și adaptarea acestuia la nevoile fiecărui utilizator.

Deși accentul principal al acestei lucrări este pus pe utilizarea inteligenței artificiale în cadrul platformei LearnAI, este important să fie prezentată și perspectiva generală asupra domeniului AI. Pentru o înțelegere mai bună a integrării și impactului acestei tehnologii, conceptele fundamentale și principalele evoluții din ultimii ani vor fi prezentate în capitolele următoare.

1.3.1. AI, machine learning și deep learning: diferențe

În primul rând, este important să înțelegem principalele diferențe dintre Inteligența Artificială (AI), Machine Learning (ML) și Deep Learning (DL):

Inteligența Artificială (AI) reprezintă un concept larg care se referă la tehnicile prin care o mașină poate imita anumite aspecte ale inteligenței umane, având capacitatea de a învăța, de a se îmbunătăți în timp și de a lua decizii pe baza unor raționamente.

Machine Learning (ML) este un subdomeniu al inteligenței artificiale care utilizează algoritmi capabili să identifice modele în date, să învețe din experiență și să ia decizii cu intervenție umană minimă.

Deep Learning (DL) este un subdomeniu al Machine Learning care utilizează rețele neuronale cu mai multe straturi pentru a procesa date într-un mod asemănător creierului uman. Modelele de Deep Learning pot recunoaște tipare complexe în imagini, sunete și texte.

Scopul principal al tehnicilor de Deep Learning este obținerea unor predicții și rezultate cât de precise. Acestea sunt utilizate frecvent pentru automatizarea sarcinilor care necesită inteligență umană, precum descrierea conținutului unei imagini, recunoașterea vorbirii, traducerea automată sau generarea de conținut textual.

1.3.2. Istoria inteligenței artificiale

Inteligența artificială a trecut prin numeroase etape de dezvoltare și cercetare pentru a ajunge la nivelul la care se află astăzi. Pentru a înțelege mai bine evoluția acestui domeniu, este important să analizăm principalele momente care au contribuit la dezvoltarea sa.

În anul 1950, Alan Turing a propus Testul Turing. Acest test este utilizat pentru a determina dacă o mașină poate demonstra un comportament inteligent asemănător celui uman. Testul implică un evaluator, o persoană și o mașină. Evaluatorul adresează întrebări atât persoanei, cât și mașinii, iar pe baza răspunsurilor trebuie să stabilească care dintre participanți este omul. Dacă evaluatorul nu poate face diferența între răspunsurile oferite de om și cele oferite de mașină, atunci se consideră că mașina a demonstrat un nivel de inteligență comparabil cu cel uman [5].

În anul 1956 a avut loc Conferința Dartmouth, considerată un punct de plecare al cercetării moderne în inteligența artificială [6].

În același an a fost dezvoltat primul program de inteligență artificială, *Logic Theorist*, creat de Allen Newell, Herbert A. Simon și Cliff Shaw [7]. Programul era conceput pentru rezolvarea problemelor și demonstrarea teoremelor matematice. Impactul său a fost semnificativ datorită capacității de a reproduce anumite forme de raționament uman și de a genera demonstrații matematice.

În 1959, Arthur Samuel a prezentat un program pentru jocul de dame care își îmbunătățește performanța prin învățare [8].

Trei ani mai târziu, Joseph Weizenbaum a creat ELIZA [9], unul dintre primele programe capabile să simuleze o conversație cu utilizatorul prin intermediul limbajului natural.

În anii următori, interesul și finanțarea cercetării în domeniul inteligenței artificiale au scăzut semnificativ, perioada fiind cunoscută sub denumirea de „AI Winter”. Aceasta reprezintă o etapă caracterizată prin reducerea investițiilor și a interesului pentru cercetarea în domeniul AI.

Ulterior, datorită progreselor tehnologice, precum procesoarele mai rapide, algoritmi îmbunătățiți și creșterea interesului academic și industrial, domeniul a început să se dezvolte din nou. În această perioadă au apărut proiecte importante, precum IBM Deep Blue, care în anul 1997 l-a învins pe campionul mondial la șah Garry Kasparov într-un meci de șase partide [10].

Un alt moment important a avut loc în anul 2011, când IBM Watson a învins campioni umani în cadrul competiției Jeopardy! [11]. Proiectul a fost coordonat de David Ferrucci, obiectivul fiind dezvoltarea unui sistem capabil să înțeleagă limbajul natural la un nivel avansat și să răspundă corect întrebărilor formulate de oameni.

Interesul pentru inteligența artificială a crescut considerabil după anul 2012, marcând începutul perioadei cunoscute sub numele de „AI Boom”. Dezvoltarea accelerată a puterii de calcul, disponibilitatea unor volume mari de date și progresul tehnicilor de Deep Learning au contribuit la extinderea rapidă a aplicațiilor bazate pe inteligență artificială.

În prezent, inteligența artificială este integrată în numeroase aspecte ale vieții cotidiene, de la vehicule autonome și sisteme de diagnostic medical până la asistenți virtuali și modele lingvistice avansate. Un exemplu relevant este ChatGPT, unul dintre cele mai cunoscute sisteme bazate pe inteligență artificială, capabil să genereze și să interpreteze limbaj natural la un nivel apropiat de cel uman.

1.3.3. Aplicații ale inteligenței artificiale

Odată cu trecerea timpului, aplicațiile bazate pe inteligență artificială au devenit din ce în ce mai prezente într-o varietate largă de domenii. Acestea sunt utilizate pentru creșterea eficienței, productivității și sustenabilității proceselor. Printre numeroasele utilizări ale inteligenței artificiale se numără colectarea datelor din surse diverse, analiza acestora pentru identificarea tiparelor, realizarea de vizualizări ale datelor și furnizarea de recomandări și informații relevante.

Printre cele mai importante domenii în care inteligența artificială este utilizată se numără astfel:

Sănătate: Prin analiza datelor pacienților, inteligența artificială poate contribui la identificarea timpurie a bolilor, poate ajuta medicii să stabilească diagnostice mai precise și poate sprijini dezvoltarea unor tratamente personalizate în funcție de nevoile fiecărui pacient.

Educație: Inteligența artificială poate fi utilizată pentru crearea unor programe de învățare personalizate, generarea automată de conținut educațional, monitorizarea progresului elevilor și studenților și adaptarea materialelor de studiu în funcție de nivelul de cunoștințe al utilizatorului.

Agricultură: Aplicațiile care analizează condițiile solului și datele meteorologice reprezintă un instrument valoros pentru fermieri. Acestea contribuie la reducerea costurilor prin automatizarea anumitor procese și monitorizarea eficientă a resurselor disponibile.

Industrie și producție: Automatizarea sarcinilor repetitive reduce costurile de producție, diminuează riscul apariției erorilor și contribuie la îmbunătățirea calității produselor finale.

Finanțe: Inteligența artificială este utilizată pentru analiza piețelor financiare, evaluarea riscurilor, detectarea fraudelor și asistarea utilizatorilor în luarea deciziilor de investiție.

Transport: AI contribuie la dezvoltarea vehiculelor autonome și la optimizarea managementului traficului, având potențialul de a reduce accidentele și de a îmbunătăți eficiența transportului.

Aceste exemple reprezintă doar o parte dintre domeniile în care inteligența artificială este utilizată în prezent, numărul aplicațiilor continuând să crească pe măsură ce tehnologia evoluează.

În contextul acestei lucrări, domeniul educațional prezintă un interes deosebit. Aplicația LearnAI utilizează inteligența artificială pentru generarea dinamică a întrebărilor și testelor, oferind utilizatorilor o experiență de învățare personalizată. Prin adaptarea conținutului educațional la preferințele și nevoile utilizatorilor, platforma demonstrează modul în care tehnologiile AI pot contribui la îmbunătățirea procesului de învățare.

1.3.4. Utilizarea inteligenței artificiale în educație și generarea conținutului

Inteligența artificială este utilizată astăzi într-o gamă largă de aplicații interactive, având rolul de a îmbunătăți experiența utilizatorilor și de a adapta conținutul la nevoile acestora. Datorită evoluției modelelor de inteligență artificială, sistemele moderne pot analiza comportamentul utilizatorilor, genera conținut nou și oferi recomandări personalizate.

Inteligența artificială este utilizată în jocuri pentru controlul agenților și generarea experiențelor interactive [12].

Conform articolului academic **AI for Game Production** [13], inteligența artificială poate fi utilizată în mai multe moduri, printre care AI ca Actor, AI ca Designer/Creator și AI ca Producător. Deși aceste categorii au fost definite inițial în contextul dezvoltării jocurilor, ele pot fi adaptate și altor tipuri de aplicații interactive.

- **AI ca Actor**

Această categorie include sistemele care interacționează direct cu utilizatorul.

Chatboții și asistenții virtuali pot răspunde întrebărilor utilizatorilor și pot oferi suport în timp real.

Modelele lingvistice moderne pot genera explicații, răspunsuri și conținut educațional adaptat contextului.

Sistemele conversaționale contribuie la creșterea gradului de interacțiune și implicare a utilizatorilor.

- **AI ca Designer / Creator**

În acest rol, inteligența artificială contribuie la proiectarea și adaptarea conținutului.

Adaptarea dificultății conținutului în funcție de nivelul utilizatorului.

Generarea automată de texte, imagini sau exerciții educaționale (Procedural Content Generation - PCG) la runtime.

Analiza performanței utilizatorilor pentru identificarea punctelor forte și a aspectelor care necesită îmbunătățiri.

Recomandarea de conținut relevant pe baza intereselor și progresului utilizatorului.

- **AI ca Producător**

În această categorie, inteligența artificială este responsabilă pentru optimizarea procesului de producție, cum ar fi generarea automată a activelor în faza de dezvoltare, testarea automată a build-urilor sau analiza automată a erorilor de către agenți software înainte de lansare.

Domeniul educațional reprezintă unul dintre cele mai importante sectoare în care aceste capacități sunt utilizate în prezent. Inteligența artificială permite dezvoltarea unor sisteme adaptive de învățare capabile să analizeze progresul utilizatorilor și să adapteze conținutul educațional în funcție de nivelul de cunoștințe, ritmul de învățare și dificultățile întâmpinate. Prin această abordare, fiecare utilizator poate beneficia de o experiență educațională personalizată.

De asemenea, inteligența artificială contribuie la automatizarea procesului de evaluare, oferind feedback rapid și identificând aspectele care necesită îmbunătățiri. Totodată, modelele moderne pot genera automat întrebări, teste, exerciții și explicații adaptate diferitelor domenii de studiu și niveluri de dificultate. Această capacitate contribuie la reducerea costurilor și a timpului necesar dezvoltării conținutului educațional, oferind în același timp o varietate mai mare de materiale de învățare.

Un alt avantaj important este posibilitatea dezvoltării tutorilor virtuali și a asistenților educaționali inteligenți, capabili să răspundă întrebărilor utilizatorilor și să ofere recomandări personalizate pe baza progresului acestora. Cu toate acestea, utilizarea inteligenței artificiale în educație ridică și anumite provocări, precum verificarea acurateții informațiilor generate, protecția datelor utilizatorilor și necesitatea monitorizării conținutului produs automat.

Aplicația LearnAI utilizează inteligența artificială în principal în rolul de AI ca Designer/Creator. Modelul GPT-4o mini este folosit pentru generarea automată a întrebărilor și testelor la runtime, personalizând conținutul în funcție de categoria selectată, nivelul de dificultate și preferințele utilizatorului. Această abordare reduce efortul necesar pentru crearea manuală a conținutului educațional, asigurând totodată un volum diversificat de materiale de studiu.

În prezent, inteligența artificială este integrată în tot mai multe aplicații software, iar utilizarea sa pentru generarea și personalizarea conținutului continuă să crească. Evoluția modelelor lingvistice și a tehnicilor de învățare automată va permite dezvoltarea unor sisteme educaționale din ce în ce mai inteligente, capabile să ofere experiențe de învățare adaptate fiecărui utilizator.

1.4. Dovezi privind utilizarea AI și a gamificării în educație

Cercetarea privind inteligența artificială în educație precede modelele generative actuale. Analiza sistematică realizată de Zawacki-Richter și colaboratorii săi a examinat 146 de articole și a identificat patru categorii principale: profilare și predicție, evaluare, sisteme adaptive

și personalizare, respectiv tutorat inteligent [14]. Autorii au remarcat și participarea redusă a educatoarelor la o parte importantă a cercetării analizate.

Meta-analiza realizată de Ma, Adesope, Nesbit și Liu a sintetizat 107 efecte, provenite din studii cu 14.321 de participanți. Sistemele de tutorat inteligent au produs rezultate mai bune decât instruirea în grup mare, materialele tipărite și alte forme de instruire asistată de calculator, fără o diferență semnificativă față de tutoratul uman individual [15]. Kulik și Fletcher au analizat 50 de evaluări controlate și au raportat rezultate favorabile în majoritatea cazurilor [16]. Aceste rezultate susțin utilitatea feedbackului și a adaptării, nu ideea că orice sistem AI este automat eficient.

Pentru modelele lingvistice generative, beneficiile trebuie analizate împreună cu riscurile. Kasneci și colaboratorii evidențiază posibilitățile de personalizare și creare a materialelor, dar și riscurile privind informația incorectă, biasul, dependența excesivă și integritatea evaluării [17]. UNESCO recomandă o abordare centrată pe om, protecția datelor și menținerea responsabilității umane asupra deciziilor educaționale [18].

Gamificarea poate susține implicarea atunci când este legată de obiective educaționale. Meta-analiza lui Sailer și Homner a identificat efecte pozitive asupra rezultatelor cognitive, motivaționale și comportamentale, cu variații determinate de context și proiectare [19]. Formatul quiz-ului este relevant și prin practica recuperării informației: Roediger și Karpicke au demonstrat că testarea repetată poate îmbunătăți retenția pe termen mai lung comparativ cu simpla recitare [20].

1.5. Analiza unor soluții existente

Moodle oferă administrarea cursurilor, evaluări și o bancă de întrebări controlată de profesor [21]. Kahoot! este orientat spre activități de quiz gamificate și oferă funcții pentru crearea asistată a conținutului [22]. Khanmigo abordează tutoratul conversațional și sprijinul acordat cursanților și profesorilor [23]. Aceste produse au obiective și niveluri de complexitate diferite, motiv pentru care comparația cu LearnAI trebuie privită ca o poziționare funcțională, nu ca demonstrarea unei superiorități generale.

LearnAI combină generarea succesivă a întrebărilor după categorie și dificultate cu progres persistent, gamification și teste create de utilizatori. Spre deosebire de o bancă fixă, conținutul poate varia de la o sesiune la alta; în schimb, utilizarea unui model generativ introduce necesitatea verificării factuale. Față de un tutor conversațional, aplicația are un scop mai restrâns: exersarea prin întrebări cu alegere multiplă și feedback imediat.

1.6. Motivația și contribuția proprie

Contribuția proprie nu constă în dezvoltarea modelului GPT sau a serviciilor Firebase. Ea constă în proiectarea și implementarea unei aplicații complete care integrează generarea contextuală, validarea structurii întrebării, progresul persistent, mecanismele de gamificare și publi-

carea testelor. Pentru testele publicate, răspunsul corect este eliminat din datele trimise browserului, iar verificarea este realizată în backend folosind o cheie protejată.

CAPITOLUL 2. ANALIZA PROBLEMEI ȘI CERINȚELE SISTEMULUI

Una dintre cele mai mari provocări în dezvoltarea platformelor educaționale este menținerea interesului utilizatorilor pe termen lung. Pentru ca o platformă să fie utilizată în mod constant, aceasta trebuie să ofere conținut relevant, variat și adaptat nevoilor fiecărui utilizator. În centrul acestei provocări se află necesitatea de a evita repetitivitatea și de a furniza permanent experiențe de învățare atractive și utile.

Acest capitol analizează principalele probleme întâlnite în dezvoltarea platformelor educaționale moderne, concentrându-se asupra diversității conținutului, costurilor de mentenanță, costurilor asociate dezvoltării și actualizării materialelor educaționale, precum și asupra dificultății menținerii implicării utilizatorilor pe termen lung.

2.1. Problema diversității conținutului în e-learning

Diversitatea conținutului reprezintă un factor esențial pentru succesul unei platforme educaționale. Utilizatorii tind să își piardă interesul atunci când primesc aceleași tipuri de întrebări sau exerciții în mod repetat. Pentru a menține un nivel ridicat de implicare, platforma trebuie să ofere conținut nou, variat și adaptat nivelului de cunoștințe al fiecărui utilizator.

În mod tradițional, crearea unui volum mare de conținut educațional necesită un efort considerabil din partea autorilor și profesorilor. Întrebările trebuie redactate, verificate și actualizate periodic pentru a rămâne relevante. Pe măsură ce numărul utilizatorilor și al domeniilor acoperite crește, această sarcină devine din ce în ce mai dificilă.

Prin urmare, este necesară identificarea unor metode care să permită generarea eficientă a conținutului educațional, fără a compromite calitatea acestuia.

2.2. Costurile de mentenanță

Costurile de mentenanță reprezintă o altă provocare importantă în dezvoltarea aplicațiilor software. După lansarea unei platforme, activitatea de dezvoltare continuă prin remedierea erorilor, implementarea actualizărilor de securitate și adaptarea aplicației la noile tehnologii și cerințe ale utilizatorilor.

În cazul platformelor online, este necesară și menținerea infrastructurii utilizate pentru stocarea datelor și furnizarea serviciilor către utilizatori. Aceste costuri includ serviciile de găzduire, bazele de date, monitorizarea aplicației și suportul tehnic.

Pentru proiectele dezvoltate de echipe mici sau de dezvoltatori individuali, costurile de mentenanță pot reprezenta o provocare semnificativă și pot influența sustenabilitatea proiectului pe termen lung.

2.3. Costurile dezvoltării conținutului

Crearea conținutului educațional de calitate reprezintă una dintre cele mai costisitoare activități în dezvoltarea unei platforme de învățare. Utilizatorii se așteaptă la materiale relevante, corecte și actualizate, iar realizarea acestora necesită timp și expertiză.

Pe măsură ce utilizatorii devin familiarizați cu conținutul existent, este necesară adăugarea continuă de noi întrebări, teste și materiale educaționale pentru a menține interesul acestora. Acest proces poate genera costuri ridicate, în special atunci când conținutul este creat exclusiv manual.

Pe măsură ce volumul conținutului educațional crește, devine necesară identificarea unor metode eficiente prin care procesul de creare și actualizare a materialelor să poată fi realizat într-un mod sustenabil, fără a compromite calitatea conținutului oferit utilizatorilor.

2.4. Dificultatea Menținerii Implicării și Motivației Utilizatorilor

Pe lângă provocările legate de diversitatea și dezvoltarea conținutului educațional, platformele de e-learning se confruntă și cu dificultatea menținerii implicării utilizatorilor pe termen lung. Deși accesul la informație este mai facil ca niciodată, numeroși utilizatori abandonează procesul de învățare după o perioadă relativ scurtă de timp, din cauza scăderii motivației sau a lipsei de interes.

Una dintre cauzele principale ale acestei probleme este caracterul repetitiv al activităților educaționale. Atunci când utilizatorii percep procesul de învățare ca fiind monoton sau lipsit de obiective clare, nivelul de implicare tinde să scadă. În plus, lipsa feedback-ului imediat și a unui sentiment de progres poate reduce motivația de a continua utilizarea platformei.

Spre deosebire de aplicațiile de divertisment și jocurile video, care utilizează frecvent sisteme de recompensare pentru a încuraja utilizarea continuă, multe platforme educaționale oferă puține mecanisme menite să stimuleze participarea activă a utilizatorilor. Din acest motiv, menținerea interesului și a motivației reprezintă una dintre cele mai importante provocări în dezvoltarea sistemelor moderne de învățare.

În lipsa unor mecanisme eficiente de motivare și recompensare, utilizatorii pot percepe procesul de învățare ca fiind monoton, ceea ce poate conduce la reducerea frecvenței utilizării platformei sau chiar la abandonarea acesteia.

2.5. Cerințe funcționale

RF1 – Utilizatorul trebuie să se poată înregistra, autentifica și deconecta prin Firebase Authentication.

RF2 – Utilizatorul trebuie să poată selecta categoria, dificultatea și numărul de întrebări al sesiunii.

RF3 – Sistemul trebuie să genereze întrebări cu exact patru variante și un singur răspuns corect.

RF4 – Sistemul trebuie să salveze profilul, XP-ul, nivelul, streak-ul și istoricul sesiunilor.

RF5 – Utilizatorul trebuie să poată crea, salva, publica și șterge teste personalizate.

RF6 – Răspunsurile testelor publicate trebuie verificate în backend, fără expunerea cheii în browser.

2.6. Cerințe nefuncționale

Securitate – cheia OpenAI nu trebuie inclusă în frontend, iar accesul la datele private trebuie limitat prin autentificare și reguli Firestore.

Corectitudinea structurii – răspunsurile AI sunt acceptate numai dacă respectă schema și regulile validatorului.

Compatibilitate – interfața trebuie să funcționeze în browsere moderne și să se adapteze la ecrane diferite.

Mentenabilitate – interfața, API-ul și integrările externe trebuie să aibă responsabilități separate.

Rezistență la erori – indisponibilitatea OpenAI sau Firebase trebuie tratată fără afișarea unor date incomplete.

2.7. Actori și flux principal

Actorul principal este utilizatorul autentificat. Acesta configurează sesiunea, răspunde la întrebări, consultă statisticile și creează teste. Frontend-ul gestionează interacțiunea, backend-ul generează și validează conținutul, OpenAI furnizează răspunsul generativ, iar Firebase gestionează identitatea și persistența datelor.

Fluxul principal începe prin selectarea parametrilor. Clientul solicită o întrebare, backend-ul construiește cererea OpenAI și validează rezultatul, iar frontend-ul afișează variantele și feedbackul. La final, rezultatul este salvat și reflectat în profil și Dashboard.

CAPITOLUL 3. PROIECTAREA ȘI ARHITECTURA SOLUȚIEI

În capitolul anterior au fost identificate principalele provocări asociate dezvoltării unei platforme educaționale moderne, printre care menținerea interesului utilizatorilor, asigurarea unei diversități ridicate a conținutului educațional, reducerea costurilor de dezvoltare și mentenanță și sincronizarea progresului utilizatorilor între mai multe dispozitive. Acest capitol prezintă soluțiile implementate în cadrul aplicației LearnAI pentru a răspunde acestor provocări.

3.1. Strategii pentru generarea și diversificarea conținutului educațional

LearnAI utilizează principii de generare procedurală pentru a produce conținut educațional nou pe baza parametrilor selectați de utilizator. În locul dezvoltării manuale a unui număr mare de întrebări și exerciții, sistemul poate genera conținut la cerere, contribuind la extinderea continuă a materialelor disponibile.

Această abordare reduce dependența de conținutul creat manual și permite scalarea platformei fără creșterea proporțională a efortului necesar dezvoltării și actualizării conținutului educațional.

3.1.1. *Generarea dinamică a întrebărilor*

Pentru reducerea repetării, LearnAI utilizează modelul GPT-4o mini pentru generarea succesivă a întrebărilor. Conținutul este produs la cerere după categoria și dificultatea selectate. Istoricul ultimelor întrebări și identificatorul răspunsului OpenAI precedent reduc probabilitatea repetării, fără a oferi o garanție absolută.

Prin utilizarea inteligenței artificiale, platforma poate genera întrebări relevante pentru numeroase domenii de studiu și niveluri de dificultate, fără a necesita actualizări manuale constante ale unei baze de date extinse de întrebări.

3.1.2. *Categorii definite de utilizator*

Pentru a oferi o experiență de învățare personalizată, LearnAI permite utilizatorilor să selecteze domeniul sau categoria pentru care doresc generarea testelor. Această funcționalitate permite adaptarea conținutului educațional la interesele și obiectivele individuale ale fiecărui utilizator.

Prin intermediul modelelor lingvistice avansate, platforma poate genera conținut pentru o gamă variată de subiecte, oferind flexibilitate și adaptabilitate sporită comparativ cu sistemele bazate pe seturi fixe de întrebări.

Prin urmare, problema diversității conținutului și a costurilor ridicate asociate dezvoltării materialelor educaționale este abordată în LearnAI prin utilizarea modelului GPT-4o mini și a tehnicilor de generare procedurală a conținutului. Aceste tehnologii permit crearea unui

volum mare de conținut educațional relevant și variat, contribuind la îmbunătățirea experienței de învățare și la creșterea sustenabilității platformei. În secțiunea următoare vor fi prezentate principalele motive pentru care modelul GPT-4o mini a fost ales ca soluție pentru cerințele aplicației LearnAI.

3.1.3. Beneficiile utilizării modelului GPT-4o mini

Modelul GPT-4o mini reprezintă componenta principală utilizată de LearnAI pentru generarea dinamică a conținutului educațional. Alegerea acestui model s-a bazat pe capacitatea sa de a înțelege contextul solicitărilor, de a genera conținut coerent și de a adapta răspunsurile în funcție de cerințele utilizatorului.

- **Capacitatea de Înțelegere a Contextului**

Unul dintre principalele avantaje ale modelului GPT-4o mini este capacitatea de a interpreta contextul unei solicitări și de a genera conținut relevant. Această caracteristică permite crearea unor întrebări adaptate categoriei și nivelului de dificultate selectate de utilizator, contribuind la diversificarea conținutului educațional și la reducerea repetitivității.

- **Baza Extinsă de Cunoștințe**

Modelul a fost antrenat pe un volum foarte mare de texte și informații provenite din surse diverse, ceea ce îi permite să genereze întrebări pentru o gamă largă de domenii de studiu. Această flexibilitate elimină necesitatea dezvoltării și menținerii unei baze de date extinse de conținut educațional creat manual și permite extinderea rapidă a materialelor disponibile în cadrul platformei.

- **Capabilități Avansate de Procesare a Limbajului Natural**

Capacitățile avansate de procesare a limbajului natural permit generarea unor texte coerente, clare și relevante din punct de vedere contextual. Acest aspect este esențial pentru formularea unor întrebări ușor de înțeles și adaptate nivelului de pregătire al utilizatorilor.

- **Generare de Conținut în Timp Real**

Un avantaj important al modelului GPT-4o mini este posibilitatea generării conținutului în timp real. Astfel, platforma poate furniza permanent întrebări noi și variate, fără a necesita actualizări manuale constante ale conținutului educațional. Această abordare contribuie la reducerea efortului necesar dezvoltării și actualizării materialelor utilizate în cadrul platformei.

- **Adaptabilitate și Scalabilitate**

Integrarea prin API permite adăugarea unor categorii fără modificarea unei bănci locale de întrebări. Scalabilitatea efectivă depinde însă de limitele serviciului OpenAI, costul apelurilor și infrastructura backend și nu a fost evaluată prin teste de încărcare.

Prin utilizarea modelului GPT-4o mini pentru generarea dinamică a întrebărilor, LearnAI răspunde provocărilor legate de diversitatea conținutului și costurile asociate dezvoltării materialelor educaționale. Capacitatea de înțelegere a contextului, baza extinsă de cunoștințe și posibilitatea generării de conținut în timp real transformă GPT-4o mini într-o soluție potrivită

pentru dezvoltarea unei platforme educaționale moderne, capabile să ofere experiențe de învățare personalizate și variate.

Modelul GPT-4o mini a fost ales în detrimentul altor soluții disponibile datorită combinației dintre capacitățile avansate de generare a textului, înțelegerea contextului și ușurința integrării prin API. Deși există și alte platforme AI, precum Google Cloud AI, IBM Watson sau Microsoft Azure AI Services, acestea sunt orientate în principal către analiza datelor, procesarea limbajului natural sau integrarea în ecosisteme specifice. Pentru cerințele aplicației LearnAI, GPT-4o mini a oferit cea mai bună combinație între flexibilitate, calitatea conținutului generat și capacitatea de personalizare a întrebărilor și testelor educaționale.

3.2. Soluții pentru reducerea costurilor de mentenanță și dezvoltare

Integrarea OpenAI poate reduce efortul redactării manuale a unui volum mare de întrebări, dar nu elimină costurile de conținut. Utilizarea API-ului, verificarea factuală, monitorizarea și tratarea rezultatelor invalide introduc costuri tehnice care trebuie luate în considerare.

Această abordare reduce considerabil volumul de muncă necesar pentru administrarea conținutului și elimină necesitatea actualizării frecvente a unor baze de date extinse de întrebări. Astfel, efortul de mentenanță se concentrează în principal pe funcționarea corectă a aplicației, gestionarea infrastructurii și actualizarea integrării cu serviciile externe utilizate.

Un alt avantaj important al utilizării API-ului OpenAI este ușurința integrării acestuia în aplicații software moderne. Documentația disponibilă, exemplele de implementare și suportul oferit dezvoltatorilor facilitează procesul de integrare și reduc timpul necesar dezvoltării funcționalităților bazate pe inteligență artificială.

În plus, modelele dezvoltate de OpenAI beneficiază de îmbunătățiri și actualizări continue, rezultate din investiții constante în cercetare și dezvoltare. Acest lucru permite aplicației LearnAI să profite de evoluția tehnologiilor de procesare a limbajului natural fără a necesita modificări majore ale arhitecturii aplicației.

Pe lângă utilizarea OpenAI, platforma beneficiază și de serviciile oferite de Firebase, care simplifică gestionarea autentificării utilizatorilor, stocarea datelor și sincronizarea progresului între dispozitive. Utilizarea unei infrastructuri gestionate reduce complexitatea dezvoltării backend-ului și contribuie la diminuarea costurilor de mentenanță pe termen lung.

Prin combinarea serviciilor OpenAI și Firebase, LearnAI reușește să ofere conținut educațional dinamic și personalizat, menținând în același timp costurile de dezvoltare și administrare la un nivel redus.

3.2.1. Avantajele Integrării Firebase

Pentru gestionarea autentificării utilizatorilor, stocarea datelor și sincronizarea progresului educațional, LearnAI utilizează platforma Firebase dezvoltată de Google. Alegerea acestei soluții s-a bazat pe ușurința integrării, infrastructura gestionată și posibilitatea dezvoltării rapide a aplicației fără implementarea unui backend complex.

- **Gestionarea Centralizată a Datelor**

Firestore pune la dispoziție serviciul Cloud Firestore, care permite stocarea securizată și organizată a datelor utilizatorilor. Prin utilizarea acestuia, LearnAI poate salva progresul educațional, statisticile, rezultatele obținute și alte informații relevante într-o bază de date accesibilă din cloud.

- **Autentificare Securizată**

Firestore Authentication oferă mecanisme moderne de autentificare care simplifică gestionarea conturilor utilizatorilor. Acest serviciu permite implementarea rapidă a procesului de autentificare și contribuie la creșterea securității aplicației.

- **Sincronizarea Datelor**

Prin stocarea informațiilor în cloud, datele utilizatorilor sunt disponibile pe orice dispozitiv pe care aceștia se autentifică. Astfel, progresul educațional este păstrat indiferent de dispozitivul utilizat, oferind o experiență consistentă și continuă.

- **Scalabilitate**

Un avantaj important al Firestore il reprezinta documentatia extinsa si instrumentele puse la dispozitia dezvoltatorilor. Acestea reduc timpul necesar implementarii functionalitatilor de autentificare si gestionare a datelor.

3.2.2. Comparatie cu alte solutii

Au fost analizate si alte alternative pentru gestionarea utilizatorilor si a datelor aplicatiei. Platforma PlayFab ofera functionalitati avansate orientate in principal catre dezvoltarea jocurilor video, multe dintre acestea nefiind necesare in contextul unei platforme educationale. Serviciile AWS ofera un grad ridicat de flexibilitate si scalabilitate, insa necesita configurare si administrare mai complexe. Dezvoltarea unui backend personalizat ar fi oferit control complet asupra infrastructurii, dar ar fi implicat costuri suplimentare de dezvoltare, mentenanta si securitate.

Din aceste motive, Firestore a reprezentat solutia cea mai potrivita pentru cerintele aplicatiei LearnAI, oferind un echilibru intre functionalitate, usurinta integrarii si costurile de dezvoltare.

3.3. Gamificarea Procesului de Învățare

Pe lângă generarea dinamică a conținutului educațional, LearnAI utilizează elemente de gamificare pentru a crește implicarea și motivația utilizatorilor. Gamificarea reprezintă procesul de integrare a mecanicilor și principiilor specifice jocurilor în contexte care nu au ca scop principal divertismentul, precum educația, sănătatea sau mediul de afaceri.

În domeniul educațional, gamificarea este utilizată pentru a transforma procesul de învățare într-o experiență mai atractivă și mai interactivă. Prin introducerea unor obiective clare, a sistemelor de recompensare și a mecanismelor de progres, utilizatorii sunt încurajați să participe activ și să își continue activitatea pe termen lung.

În contextul aplicației LearnAI, elementele de gamificare au rolul de a combate scăderea motivației și abandonarea procesului de învățare, probleme identificate în capitolul anterior. Prin utilizarea nivelurilor, punctelor de experiență, streak-urilor și clasamentelor, platforma urmărește să ofere utilizatorilor un sentiment constant de progres și realizare.

Secțiunile următoare prezintă beneficiile utilizării gamificării în educație și mecanismele implementate în cadrul platformei LearnAI pentru creșterea implicării utilizatorilor.

3.3.1. Beneficiile Gamificării în Educație

Gamificarea a devenit o metodă tot mai utilizată în domeniul educațional datorită capacității sale de a crește implicarea și motivația utilizatorilor. Prin integrarea unor mecanisme inspirate din jocuri, procesul de învățare poate deveni mai atractiv și mai interactiv, contribuind la îmbunătățirea experienței educaționale.

Unul dintre principalele beneficii ale gamificării este creșterea motivației utilizatorilor. Sistemele de recompensare, nivelurile și realizările oferă obiective clare și încurajează utilizatorii să își continue activitatea pentru a obține noi rezultate și recompense. Acest lucru contribuie la menținerea interesului pe termen lung și la reducerea riscului de abandon al procesului de învățare.

Gamificarea contribuie și la creșterea implicării active a utilizatorilor. În loc să participe pasiv la procesul educațional, aceștia sunt încurajați să își urmărească progresul, să îndeplinească obiective și să își îmbunătățească performanțele. Astfel, procesul de învățare devine mai dinamic și mai interactiv.

Un alt avantaj important îl reprezintă feedback-ul imediat. Prin utilizarea punctelor de experiență, a nivelurilor și a altor indicatori de progres, utilizatorii pot observa rapid rezultatele activității lor și pot identifica aspectele care necesită îmbunătățiri. Acest lucru contribuie la dezvoltarea unui sentiment de progres și realizare.

De asemenea, mecanismele competitive, precum clasamentele și comparația performanțelor între utilizatori, pot stimula participarea și pot încuraja atingerea unor rezultate mai bune. Atunci când sunt implementate într-un mod echilibrat, aceste elemente pot crește gradul de implicare fără a afecta obiectivul principal al platformei, și anume procesul de învățare.

Prin urmare, gamificarea reprezintă o metodă eficientă de creștere a motivației, implicării și retenției utilizatorilor în cadrul platformelor educaționale. Din acest motiv, LearnAI integrează mai multe mecanisme inspirate din jocuri pentru a transforma procesul de învățare într-o experiență mai atractivă și mai motivantă.

3.3.2. Niveluri, XP și Streak-uri, Clasamente și Sisteme de Recompensare

Pentru a valorifica beneficiile gamificării și pentru a menține implicarea utilizatorilor pe termen lung, LearnAI integrează mai multe mecanisme inspirate din jocuri. Acestea au rolul de a oferi feedback constant, de a evidenția progresul utilizatorilor și de a încuraja utilizarea regulată a platformei.

Unul dintre mecanismele implementate este sistemul de puncte de experiență (XP). Utilizatorii primesc puncte pentru finalizarea testelor și pentru obținerea unor rezultate bune, ceea ce le permite să urmărească în mod clar progresul realizat pe parcursul procesului de învățare.

Pe baza punctelor de experiență acumulate, utilizatorii avansează prin diferite niveluri. Sistemul de niveluri oferă obiective pe termen lung și creează un sentiment de progres continuu, încurajând utilizatorii să își îmbunătățească performanțele și să utilizeze platforma în mod constant.

LearnAI utilizează și un sistem de streak-uri, care evidențiază numărul de zile consecutive în care utilizatorul desfășoară activități educaționale pe platformă. Acest mecanism încurajează dezvoltarea unor obiceiuri constante de învățare și contribuie la creșterea retenției utilizatorilor.

Pentru a introduce o componentă socială și competitivă, platforma include clasamente care permit compararea performanțelor între utilizatori. Acestea stimulează participarea activă și oferă un motiv suplimentar pentru îmbunătățirea rezultatelor obținute.

Pe lângă aceste mecanisme, LearnAI utilizează și diverse sisteme de recompensare, menite să ofere utilizatorilor recunoaștere pentru progresul și realizările lor. Împreună, aceste elemente contribuie la transformarea procesului de învățare într-o experiență mai atractivă, mai interactivă și mai motivantă.

Prin integrarea nivelurilor, punctelor de experiență, streak-urilor, clasamentelor și sistemelor de recompensare, LearnAI urmărește să crească motivația utilizatorilor și să încurajeze utilizarea constantă a platformei, contribuind astfel la îmbunătățirea procesului de învățare.

CAPITOLUL 4. Implementarea și descrierea aplicației

În acest capitol sunt prezentate aspectele tehnice care au stat la baza dezvoltării aplicației LearnAI, precum și tehnologiile utilizate pentru implementarea acesteia. Sunt descrise arhitectura sistemului, componentele principale ale aplicației și modul în care acestea interacționează pentru a oferi utilizatorilor o experiență de învățare personalizată.

Aplicația LearnAI combină servicii de inteligență artificială, tehnologii web moderne și servicii cloud pentru a genera conținut educațional dinamic, a gestiona autentificarea utilizatorilor și a stoca progresul acestora. Soluția utilizează modelul GPT-4o mini prin intermediul API-ului OpenAI pentru generarea întrebărilor și Firebase pentru gestionarea utilizatorilor și a datelor asociate acestora.

Pentru o înțelegere mai clară a modului de funcționare al aplicației, figura următoare prezintă arhitectura generală a sistemului și principalele componente care contribuie la funcționarea platformei LearnAI.

4.1. Arhitectura Generală a Sistemului

Aplicația LearnAI este alcătuită dintr-un frontend dezvoltat cu React și TypeScript, un backend ASP.NET Core și două servicii externe: OpenAI și Firebase. Legăturile dintre aceste componente sunt prezentate în Figura 4.1. Frontend-ul gestionează interfața și interacțiunile utilizatorului. Backend-ul primește cererile pentru generarea întrebărilor, comunică cu OpenAI și validează răspunsurile înainte de a le transmite aplicației. Firebase este utilizat pentru autentificare și pentru stocarea profilului, progresului, statisticilor și testelor. Această separare protejează cheia OpenAI și permite organizarea clară a responsabilităților fiecărei componente.

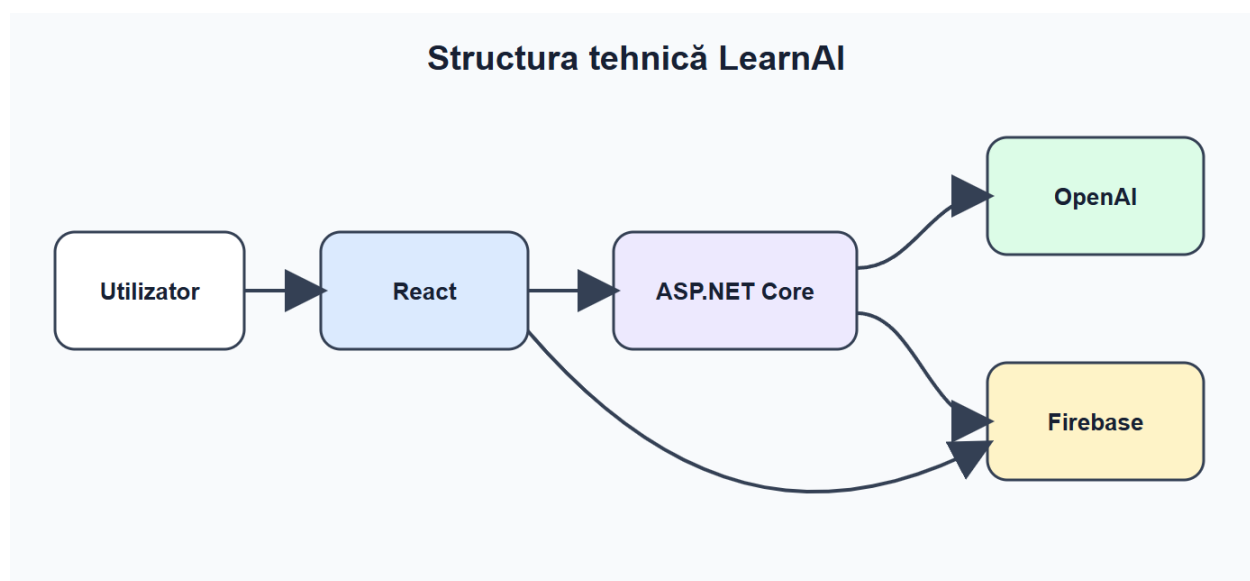


Figura 4.1. Arhitectura generală a sistemului LearnAI

4.2. Designul Interfeței Utilizator

Interfața aplicației LearnAI este implementată cu React și TypeScript, iar aspectul vizual este controlat prin CSS. Ecranele sunt afișate în funcție de starea aplicației și de acțiunile utilizatorului: autentificare, dashboard, configurarea unui test, rezolvarea întrebărilor, statistici sau editarea profilului. Această abordare permite reutilizarea elementelor vizuale și păstrarea unui stil unitar.

Pentru pictogramele de navigare și pentru diferite acțiuni sunt utilizate elemente SVG. Fiind bazate pe vectori, acestea își păstrează claritatea la dimensiuni diferite și pot fi adaptate vizual prin CSS. Interfața include și animații Lottie, stocate în format JSON, utilizate pentru încărcarea întrebărilor, afișarea rezultatelor și evidențierea unor acțiuni importante. Utilizarea acestui format bazat pe text evită introducerea unor fișiere video voluminoase.

O componentă distinctă a interfeței este sistemul de avataruri. Avatarul este generat și personalizat folosind stiluri DiceBear, precum Avataaars, Lorelei, Notionists și Open Peeps. Utilizatorul poate modifica stilul, fundalul, părul, expresia, accesoriile și alte caracteristici vizuale. Unele opțiuni sunt condiționate de nivelul obținut, ceea ce leagă personalizarea avatarului de sistemul de progres al aplicației. Configurația aleasă este salvată în profilul utilizatorului și este reutilizată în dashboard și clasament.

Interfața oferă teme luminoasă și întunecată, implementate prin modificarea claselor și a proprietăților CSS. Componentele sunt responsive și își adaptează poziția și dimensiunea în funcție de spațiul disponibil. Feedbackul vizual este oferit prin culori, stări ale butoanelor, bare de progres și mesaje afișate după acțiunile utilizatorului.

4.2.1. Pagina de Autentificare

Pagina de autentificare reprezintă punctul de acces la funcțiile aplicației LearnAI și permite atât conectarea la un cont existent, cât și înregistrarea unui utilizator nou. Interfața este implementată în React, iar valorile introduse în câmpurile pentru adresa de e-mail și parolă sunt păstrate temporar în starea componentei. Trimiterea formularului declanșează operația corespunzătoare fără reîncărcarea completă a paginii.

Autentificarea este realizată prin Firebase Authentication, folosind metoda bazată pe e-mail și parolă. Pentru înregistrare este apelată funcția de creare a utilizatorului, iar pentru conectare este utilizată funcția de autentificare cu datele introduse. Parola nu este stocată în baza de date a aplicației și nu este transmisă backend-ului ASP.NET Core, fiind procesată direct de serviciul Firebase Authentication.

După autentificarea reușită, Firebase creează o sesiune asociată utilizatorului și furnizează un obiect care conține identificatorul unic al contului. Aplicația monitorizează modificarea stării de autentificare și actualizează interfața atunci când utilizatorul se conectează sau se deconectează. Astfel, accesul la dashboard și la celelalte ecrane este permis numai după determinarea stării curente a sesiunii.

Identificatorul Firebase este utilizat pentru asocierea contului cu documentele salvate în Cloud Firestore. La prima autentificare, aplicația poate crea automat profilul inițial al utilizatorului, care conține valori implicite pentru numele afișat, avatar, nivel, puncte XP, streak și statistici. La autentificările următoare, aceste informații sunt citite din Firestore și încărcate în starea aplicației. În acest mod, progresul nu depinde de dispozitivul sau browserul folosit.

Pentru cererile către endpoint-urile protejate ale backend-ului, precum operațiile asupra testelor personalizate, aplicația obține tokenul de identificare Firebase al utilizatorului autentificat. Tokenul este introdus în antetul HTTP Authorization, folosind formatul Bearer. Backend-ul poate identifica astfel utilizatorul care a inițiat cererea, iar serviciul Firestore respinge cererile care nu conțin o autentificare validă.

Pagina gestionează și stările care pot apărea în timpul autentificării. Pe durata procesării formularului, butoanele pot fi dezactivate pentru evitarea trimiterilor multiple. Dacă adresa de e-mail este invalidă, parola nu respectă cerințele Firebase, contul nu există sau serviciul extern nu poate fi contactat, interfața afișează un mesaj de eroare. Mesajele tehnice sunt transformate într-o formă mai ușor de înțeles pentru utilizator.

Din punct de vedere vizual, formularul este integrat în stilul general LearnAI. Pictogramele SVG sunt utilizate pentru elementele de interfață deoarece pot fi redimensionate fără pierderea clarității. Structura paginii este adaptată prin reguli CSS responsive, astfel încât formularul să rămână lizibil atât pe desktop, cât și pe dispozitive cu ecran redus.

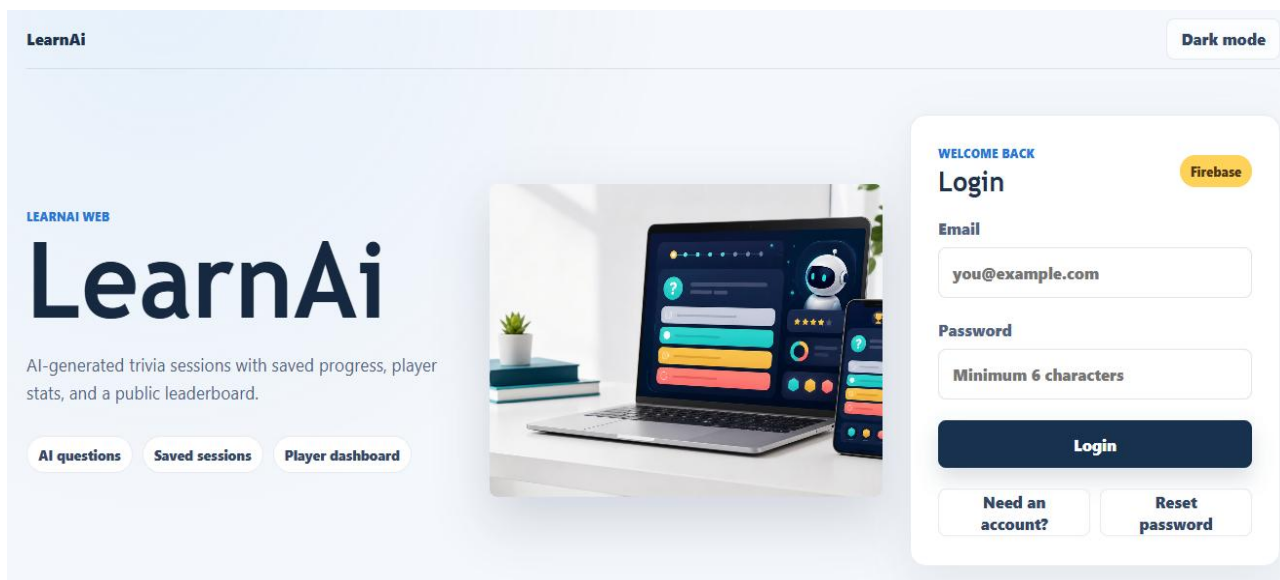


Figura 4.2. Pagina de autentificare a aplicației LearnAI.

4.2.2. Dashboard-ul Utilizatorului

Dashboard-ul este ecranul dedicat statisticilor utilizatorului. Acesta nu este afișat automat după autentificare, ci poate fi accesat din bara de navigare a aplicației. Ecranul este controlat în React prin valoarea dashboard a stării care stabilește pagina activă.

În partea superioară este afișat titlul „Your stats”, alături de componenta BrainAnimation. Aceasta utilizează o animație Lottie pentru a diferenția vizual pagina de celelalte secțiuni ale aplicației. Animația are un rol vizual și nu influențează calcularea statisticilor.

Informațiile sunt preluate din profilul deja încărcat în starea aplicației. React transmite valorile componentelor vizuale, iar interfața este actualizată automat atunci când profilul se modifică după finalizarea unui test sau revendicarea unei recompense.

Statisticile principale sunt organizate într-un grid de panouri informaționale. Dashboard-ul afișează următoarele valori:

- nivelul curent;
- punctele XP disponibile la nivelul curent;
- cel mai bun scor general;
- numărul testelor finalizate;
- streak-ul curent;
- cel mai bun streak;
- acuratețea generală;
- cea mai bună acuratețe obținută într-un test.

Acuratețea generală este calculată pe baza numărului total de răspunsuri corecte și a numărului total de întrebări la care utilizatorul a răspuns. Dacă nu există încă răspunsuri salvate, interfața afișează valoarea 0%, evitând o împărțire la zero.

Separat de grid-ul principal sunt afișate totalul răspunsurilor corecte și data ultimei activități. Numărul răspunsurilor este prezentat sub forma raportului dintre răspunsurile corecte și totalul întrebărilor rezolvate.

A doua zonă a Dashboard-ului este destinată istoricului recent. Pentru fiecare sesiune sunt afișate dificultatea, categoria, procentul de răspunsuri corecte, numărul efectiv al răspunsurilor corecte, scorul general și data finalizării.

Acuratețea fiecărei sesiuni este calculată separat. Aceasta este obținută prin împărțirea răspunsurilor corecte la numărul întrebărilor rezolvate și rotunjirea rezultatului procentual. Prin această abordare, rezultatele testelor cu lungimi diferite pot fi comparate mai ușor.

Lista sesiunilor este construită din obiectele existente în starea recentSessions. React parcurge lista și generează câte un element vizual pentru fiecare sesiune. Identificatorul sesiunii este folosit drept cheie React, pentru actualizarea eficientă a listei.

Dashboard-ul include și un buton Refresh. Acesta permite reîncărcarea profilului și a sesiunilor recente. Pe durata actualizării, interfața existentă rămâne afișată, iar valorile sunt înlocuite după finalizarea operației.

Dacă utilizatorul nu a terminat încă niciun test, lista nu rămâne goală fără explicație. În locul acesteia este afișat mesajul „No completed sessions yet.”, care reprezintă starea inițială a componentei.

Din punct de vedere vizual, statisticile folosesc casete informaționale (elemente de tip card) CSS cu titluri scurte și valori numerice evidențiate. Pictogramele și animația Lottie sunt separate de conținutul statistic, astfel încât informațiile să rămână lizibile.

Structura Dashboard-ului este responsive. Pe ecranele mari, aceste elemente sunt distribuite pe mai multe coloane, iar pe dispozitivele cu lățime redusă sunt reorganizate vertical. Lista sesiunilor își adaptează, de asemenea, coloanele pentru a păstra lizibilitatea valorilor.

Organizarea statisticilor generale ale utilizatorului este prezentată în Figura 4.3.

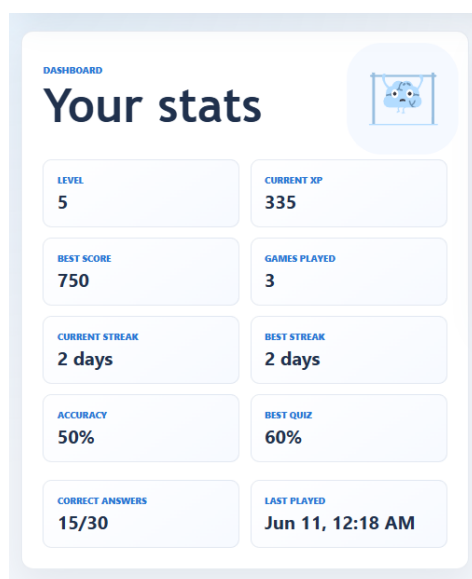


Figura 4.3. Dashboard-ul cu statisticile generale ale utilizatorului.

Istoricul sesiunilor finalizate este prezentat în Figura 4.4.

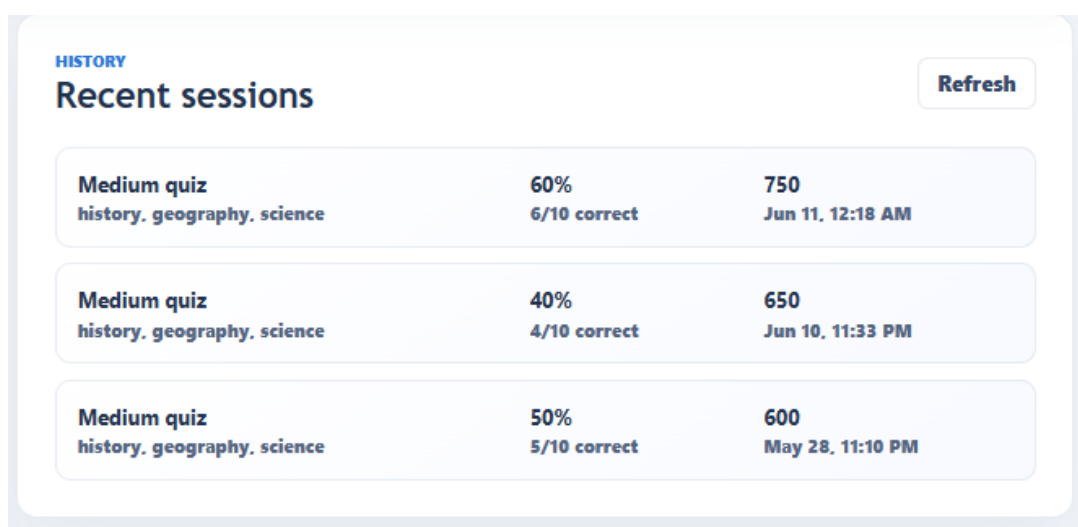


Figura 4.4. Istoricul recent al sesiunilor finalizate.

4.2.3. Configurarea Testelor

Ecranul de configurare permite stabilirea parametrilor utilizați pentru generarea unei sesiuni de întrebări. Înainte de începerea testului, utilizatorul poate selecta categoria, nivelul de dificultate și numărul de întrebări. Valorile alese sunt gestionate în starea componentei React și sunt validate înaintea inițializării sesiunii.

Aplicația pune la dispoziție o listă de categorii predefinite, dar permite și introducerea unui subiect personalizat. Această opțiune extinde domeniul întrebărilor dincolo de o listă fixă. Categoria selectată este transmisă backend-ului și inclusă în instrucțiunile trimise modelului OpenAI. Deoarece modelele lingvistice pot interpreta subiecte formulate liber, utilizatorul poate solicita întrebări pentru domenii mai specifice.

Nivelul de dificultate poate fi ales dintre variantele ușor, mediu și greu. Valoarea nu modifică un algoritm local de selectare a întrebărilor, ci este inclusă în cererea de generare. Modelul OpenAI formulează întrebarea în funcție de dificultatea solicitată. Caracterul dificultății rămâne dependent de interpretarea modelului, motiv pentru care această proprietate trebuie analizată separat în cadrul unei evaluări a conținutului generat.

Numărul de întrebări determină durata sesiunii și este utilizat pentru calcularea progresului. La pornirea testului, aplicația inițializează indexul întrebării curente, scorul, lista întrebărilor deja utilizate și starea metodelor de ajutor. Aceste date sunt păstrate temporar în frontend și sunt resetate la începutul fiecărei sesiuni.

După apăsarea butonului de pornire, clientul trimite o cerere HTTP de tip POST către endpoint-ul `/api/trivia/question`. Corpul cererii conține categoria, dificultatea și informațiile necesare evitării întrebărilor repetitive. Backend-ul construiește solicitarea către OpenAI și returnează întrebarea numai după parsarea și validarea răspunsului.

Pentru prima întrebare, istoricul sesiunii este gol. După fiecare răspuns, textul întrebării este adăugat în lista întrebărilor utilizate. La generările următoare, lista este transmisă backend-ului împreună cu identificatorul răspunsului OpenAI precedent. Acest mecanism oferă modelului contextul sesiunii și reduce probabilitatea repetării unor întrebări identice sau foarte asemănătoare.

În timpul obținerii primei întrebări, interfața afișează o stare de încărcare realizată cu ajutorul unei animații Lottie. Dacă serviciul OpenAI nu este disponibil sau răspunsul nu respectă structura necesară, utilizatorul este direcționat către o pagină de eroare controlată și poate relua operația. Astfel, o eroare externă nu conduce la afișarea unui ecran incomplet.

Din punct de vedere vizual, opțiunile sunt organizate în controale și elemente de tip card CSS, iar selecția activă este evidențiată prin culoare și modificarea stilului elementului. Pictogramele SVG sunt utilizate pentru identificarea acțiunilor, iar structura responsive permite reorganizarea opțiunilor pe ecrane cu lățime redusă.

Parametrii disponibili pentru configurarea unei sesiuni sunt prezentați în Figura 4.5.

The image shows a web interface for configuring a quiz session in LearnAI. At the top, there are three status boxes: 'SIGNED IN AS altcineva', 'SAVED LEVEL 5', and 'SAVED XP 335'. Below these are three buttons: 'AI questions', 'One-use helpers', and 'Fast preloading'. A 'Name' field contains the text 'User'. The 'Categories' section shows a list of selected categories: 'Board Games, Pop Culture Trivia, Unexplained Mysteries, Famous Landmarks'. Below this is a button 'Pick combined categories'. A section titled 'Browse all categories' contains a list of category buttons: 'World Capitals', 'Famous Landmarks' (highlighted in green), 'Movie Quotes', 'Science Fiction Films', 'Classical Literature', 'Mythology', 'U.S. Presidents', 'Nobel Prize Winners', 'Olympic Records', 'Famous Artists', 'Musical Instruments', and 'Constellations'. There is a 'Clear' button and a scroll bar. Below the categories, there are two dropdown menus: 'Difficulty' set to 'Medium' and 'Duration' set to 'Custom'. A 'Number of questions' field is set to '13'. At the bottom, there are two buttons: '13 questions' and 'Medium', and a large dark blue button labeled 'Start game'.

Figura 4.5. Configurarea unei sesiuni de întrebări în LearnAI.

4.2.4. Ecranul de Rezolvare a Testelor

Ecranul de rezolvare reprezintă componenta principală a sesiunii de testare. Acesta afișează întrebarea curentă, cele patru variante de răspuns, progresul sesiunii și metodele de ajutor disponibile. Datele întrebării sunt primite de la backend într-o structură JSON validată înainte de afișare.

Pentru întrebările generate prin OpenAI, structura conține textul întrebării, un indiciu și o listă cu exact patru răspunsuri. Fiecare variantă conține textul afișat și informația necesară verificării răspunsului. Validatorul backend respinge întrebările care nu conțin exact patru variante sau care au mai multe răspunsuri marcate drept corecte.

Întrebarea curentă este păstrată în starea componentei React. La selectarea unei variante, aplicația înregistrează opțiunea utilizatorului și actualizează stilul butonului corespunzător. După confirmare, răspunsul selectat este comparat cu răspunsul corect, iar interfața afișează imediat

rezultatul. Pentru evitarea selectării mai multor variante, răspunsurile sunt dezactivate după efectuarea alegerii.

Feedbackul este reprezentat prin culori și stări CSS distincte. Varianta corectă este evidențiată vizual, iar o variantă incorectă selectată este marcată separat. Această diferențiere permite utilizatorului să identifice rapid atât rezultatul, cât și răspunsul care trebuia ales. După afișarea feedbackului, utilizatorul poate trece la întrebarea următoare.

Progresul testului este calculat pe baza indexului întrebării curente și a numărului total stabilit în etapa de configurare. Interfața afișează poziția în sesiune și o bară de progres. Spre deosebire de unele aplicații de trivia, LearnAI nu impune un cronometru strict, astfel încât utilizatorul poate analiza întrebarea în propriul ritm.

Aplicația oferă trei metode de ajutor. Opțiunea 50/50 elimină două variante incorecte, opțiunea Hint afișează indiciul generat împreună cu întrebarea, iar opțiunea Reveal evidențiază răspunsul corect. Starea fiecărei metode este păstrată pentru sesiunea curentă, astfel încât aceasta să nu poată fi reutilizată după consumare. La începerea unui test nou, stările sunt resetate.

Pentru testele personalizate publicate este utilizat un mecanism diferit de verificare. Răspunsul corect nu este inclus în datele trimise browserului. Clientul transmite către backend identifiatorul testului, indexul întrebării și indexul variantei selectate. Backend-ul verifică răspunsul folosind cheia protejată și returnează numai rezultatul verificării. Astfel, răspunsurile testelor publicate nu pot fi identificate prin inspectarea datelor primite de frontend.

La finalul sesiunii sunt calculate scorul și punctele XP obținute. Rezultatul este salvat în Firestore și contribuie la actualizarea nivelului, statisticilor și clasamentului. Interfața finală poate utiliza animații Lottie pentru evidențierea rezultatului, precum animația de confetti sau animația asociată câștigătorului.

Componentele vizuale sunt organizate într-un panou responsive. Butoanele răspunsurilor își modifică dimensiunea și poziția în funcție de lățimea ecranului, iar pictogramele SVG permit păstrarea clarității metodelor de ajutor pe dispozitive diferite.

Elementele disponibile în timpul rezolvării unei întrebări sunt prezentate în Figura 4.6.

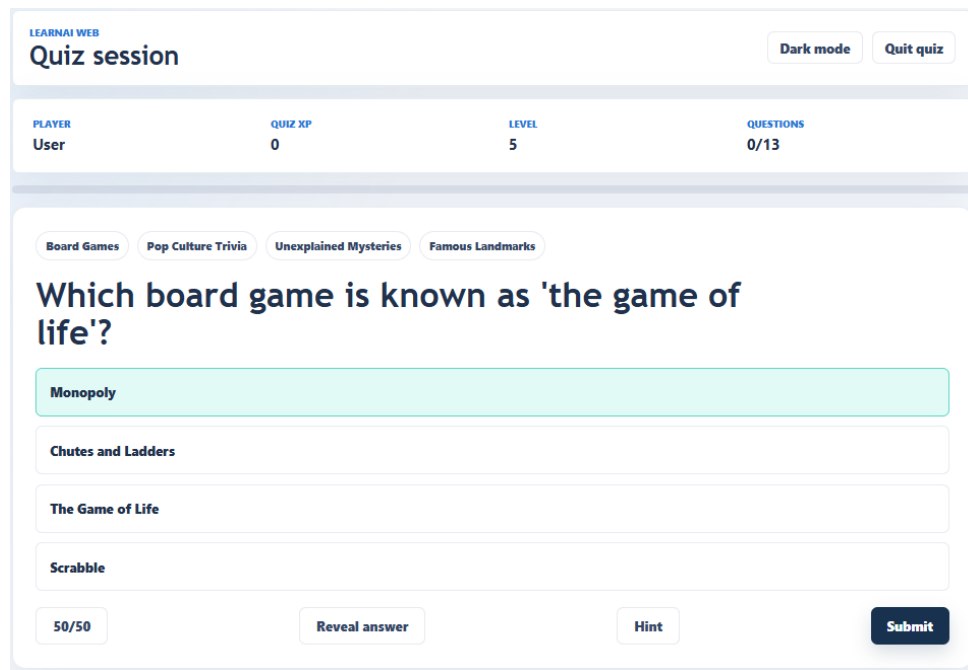


Figura 4.6. Ecranul de rezolvare a unei întrebări generate.

Feedbackul oferit după selectarea unui răspuns este prezentat în Figura 4.7.

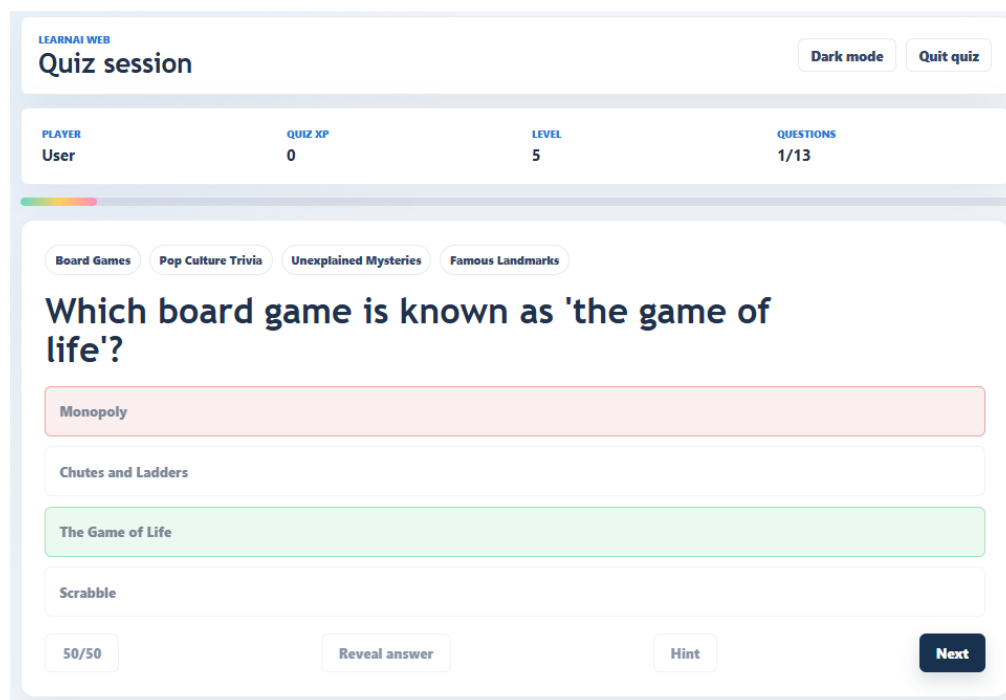


Figura 4.7. Feedbackul vizual afișat după verificarea răspunsului.

4.2.5. Setări și profil utilizator

Pagina Settings este destinată configurării identității vizuale și a preferințelor contului. Aceasta poate fi accesată din bara de navigare sau prin butonul Account disponibil în pagina de configurare a testului.

Afișarea paginii este controlată prin valoarea settings a stării appView. Formularul utilizează starea settingsForm, care conține numele afișat și configurația curentă a avatarului. Modificările sunt prezentate în interfață înainte de salvarea lor.

Câmpul destinat numelui este implementat ca element controlat React. Valoarea afișată provine din starea formularului, iar fiecare modificare actualizează obiectul settingsForm. Dacă utilizatorul introduce numai spații sau lasă câmpul gol, aplicația folosește un nume implicit derivat din cont.

Avatarul este afișat într-o zonă de previzualizare prin componenta LocalAvatar. Aceasta generează adresa imaginii pe baza configurației selectate. La modificarea unei proprietăți, React actualizează starea și regenerează imediat previzualizarea.

Interfața permite selectarea mai multor stiluri DiceBear:

- Avataaars;
- Lorelei;
- Open Peeps;
- Notionists.

Fiecare stil pune la dispoziție propriile categorii de personalizare. În funcție de stil, utilizatorul poate modifica fundalul, coafura, ochii, sprâncenele, gura, hainele, accesoriile și alte elemente grafice.

Avatarurile sunt furnizate în format SVG. Această alegere permite afișarea lor atât în previzualizarea mare din Settings, cât și în dimensiunile reduse utilizate în bara de navigare și clasament, fără pierderea clarității.

Pentru schimbarea opțiunilor sunt utilizate butoane de navigare înainte și înapoi. Există și o funcție Randomize avatar, care generează automat o combinație folosind numai opțiunile disponibile pentru nivelul curent.

Unele stiluri și elemente sunt condiționate de nivel. Fiecare opțiune poate conține proprietatea minLevel. Interfața compară această valoare cu nivelul profilului și afișează un mesaj atunci când elementul nu este încă disponibil.

Utilizatorul poate previzualiza anumite elemente blocate, dar nu poate salva configurația până când acestea nu sunt deblocate. Butonul de salvare este dezactivat dacă formularul conține un stil sau un element care necesită un nivel superior.

La trimiterea formularului este apelată funcția handleSettingsSubmit. Pe durata salvării, butonul este dezactivat și afișează mesajul Saving.... După finalizarea operației, este prezentat un mesaj de succes sau de eroare.

Pagina Settings include și controlul pentru alegerea temei. Acesta este implementat ca un comutator care modifică starea darkMode. Pentru reprezentarea temei luminoase și întunecate sunt utilizate pictograme SVG pentru soare și lună.

Comutatorul include attribute precum `role="switch"` și `aria-pressed`, pentru descrierea stării sale către tehnologiile asistive. Modificarea temei schimbă clasele CSS aplicate interfeței, fără reconstruirea componentelor React.

În partea inferioară este afișat un rezumat al progresului salvat. Acesta conține nivelul, cel mai bun scor, streak-ul

curent și acuratețea generală. Rezumatul nu înlocuiește Dashboard-ul, ci permite verificarea rapidă a profilului în timpul modificării setărilor.

Avatarul și numele salvate sunt reutilizate în bara de navigare, pagina de configurare, Dashboard și clasament. Actualizarea stării profilului produce astfel o modificare vizuală în toate componentele care afișează identitatea utilizatorului.

Opțiunile pentru modificarea numelui și personalizarea avatarului sunt prezentate în Figura 4.8.

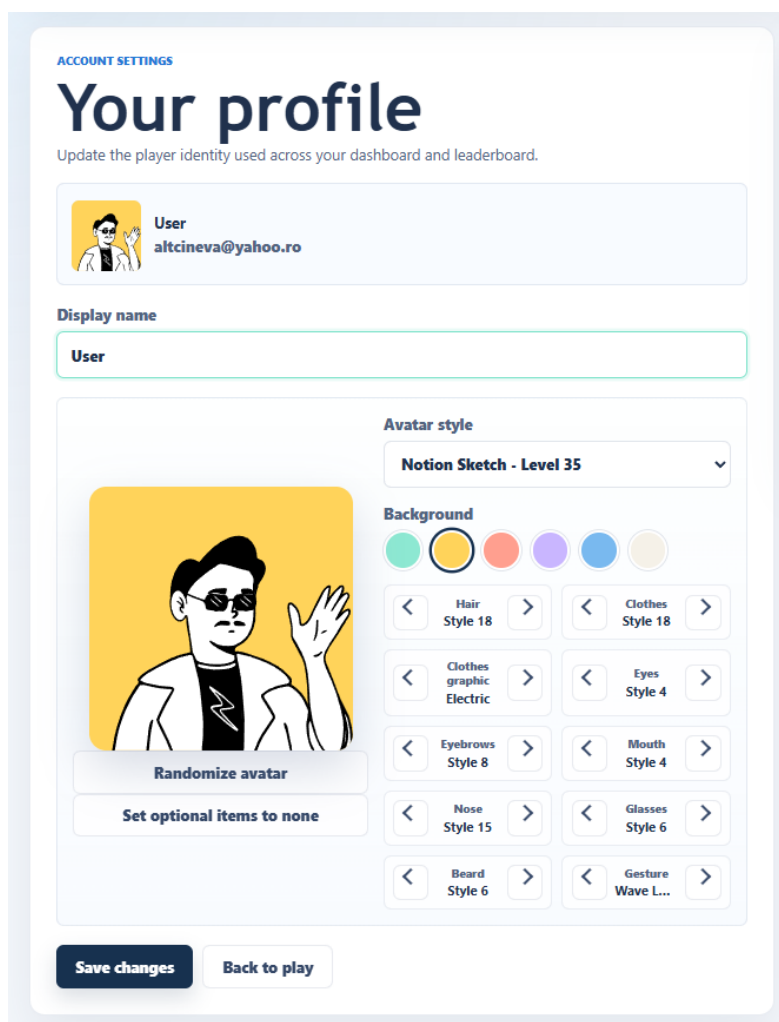


Figura 4.8. Configurarea numelui și a avatarului în pagina Settings.
Setarea temei și rezumatul progresului sunt prezentate în Figura 4.9.

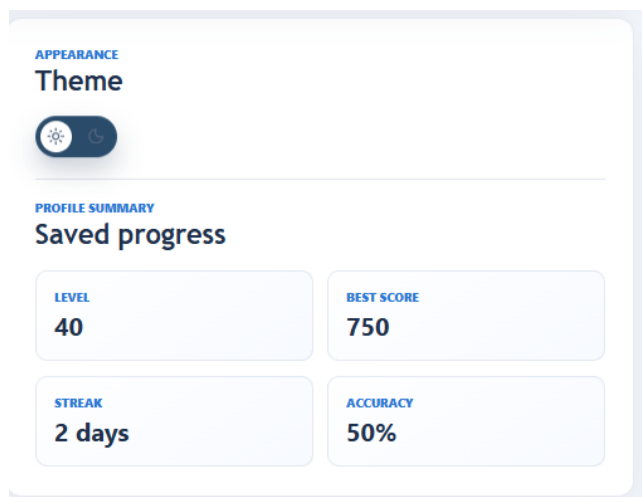


Figura 4.9. Preferințele vizuale și rezumatul profilului utilizatorului.

4.3. Implementare tehnică

Implementarea LearnAI este împărțită în frontend, backend și servicii externe. Frontend-ul este dezvoltat cu React și TypeScript, backend-ul utilizează ASP.NET Core 9, iar autentificarea și stocarea datelor sunt realizate cu Firebase.

Generarea întrebărilor este asigurată prin OpenAI Chat Completions API.

Clientul React gestionează starea interfeței, navigarea între ecrane și desfășurarea sesiunilor de testare. Vite este utilizat pentru dezvoltarea locală și generarea build-ului de producție. TypeScript definește structurile datelor folosite pentru profiluri, întrebări, răspunsuri, statistici și teste personalizate.

Backend-ul expune endpoint-uri HTTP pentru generarea întrebărilor și administrarea testelor personalizate. Acesta construiește cererile către OpenAI, verifică structura răspunsurilor și protejează informațiile care nu trebuie să ajungă în browser, precum cheia API și răspunsurile corecte ale testelor publicate.

Firebase Authentication gestionează identitatea utilizatorilor, iar Cloud Firestore păstrează profilurile, progresul, clasamentul și testele. Comunicarea dintre componente se realizează prin cereri HTTP și structuri JSON.

În subcapitolele următoare sunt prezentate separat implementarea frontend-ului, backend-ului și integrarea serviciilor Firebase și OpenAI.

4.3.1. Implementarea frontend-ului cu React și TypeScript

Frontend-ul aplicației LearnAI este dezvoltat cu React și TypeScript și se află în proiectul TrivAi.Client. Pentru dezvoltare și generarea versiunii de producție este utilizat Vite. Acesta gestionează pornirea serverului local, procesarea fișierelor și construirea pachetului care va fi executat în browser.

Structura principală a proiectului client este următoarea:

```
TrivAi.Client
├─ src
│   ├── assets
│   ├── components
│   │   └─ ui
│   ├── firebase.ts
│   ├── main.tsx
│   └─ styles.css
├─ package.json
└─ vite.config.ts
```

Fișierul `main.tsx` reprezintă punctul principal al interfeței. Acesta inițializează aplicația React și coordonează ecranele pentru autentificare, dashboard, configurarea testelor, rezolvarea întrebărilor, statistici, clasament și profil. Fișierul `firebase.ts` conține tipurile și funcțiile folosite pentru comunicarea cu Firebase, iar `styles.css` definește stilurile generale, temele și regulile responsive.

Componentele vizuale reutilizabile sunt organizate în directorul `components/ui`. Aici se află componente pentru animația de încărcare a întrebărilor, animația de confetti, afișarea rezultatului, tratarea erorilor de conexiune și navigarea principală. Separarea acestor elemente reduce repetarea codului și permite utilizarea lor în mai multe ecrane.

Gestionarea stării este realizată cu hook-uri React. Valorile păstrate în stare includ utilizatorul autentificat, profilul încărcat, ecranul curent, configurația testului, întrebarea curentă, scorul, progresul sesiunii și starea metodelor de ajutor. Modificarea unei valori determină React să actualizeze numai componentele afectate.

Navigarea nu necesită reîncărcarea completă a paginii. Ecranul afișat este controlat printr-o valoare de stare care indică zona activă a aplicației. Acțiunile utilizatorului modifică această valoare, iar React afișează componenta corespunzătoare. Starea testului este păstrată separat, astfel încât navigarea și desfășurarea unei sesiuni să poată fi controlate independent.

TypeScript este utilizat pentru definirea structurii datelor manipulate în frontend. Sunt definite tipuri pentru

profil, avatar, sesiuni, clasament, întrebări și teste personalizate. De exemplu, configurația avatarului stabilește proprietățile disponibile pentru stil, fundal, păr, expresie și accesorii. Utilizarea tipurilor permite identificarea multor erori înainte de rularea aplicației.

Formularele sunt implementate sub forma unor componente controlate. Valoarea fiecărui câmp este asociată unei variabile de stare, iar modificările sunt tratate prin evenimente React. Această abordare este utilizată pentru autentificare, configurarea testelor, modificarea profilului și crearea testelor personalizate. Înaintea trimiterii, valorile pot fi verificate, iar eventualele erori sunt afișate în apropierea formularului.

Aspectul vizual este definit prin clase CSS. Fișierul de stil conține reguli pentru casete, panouri, butoane, formulare, bare de progres și organizarea responsive a paginilor. Tema luminoasă și tema întunecată folosesc aceeași structură a componentelor, dar modifică paleta de culori și contrastul.

Pictogramele interfeței sunt implementate în format SVG. Elementele SVG pot fi redimensionate fără pierderea clarității și pot prelua culori din stilurile CSS. Acestea sunt utilizate în navigare, în butoanele de acțiune și în indicatorii de stare.

Pentru feedback vizual sunt utilizate animații Lottie. Acestea sunt definite prin fișiere JSON și sunt redatate în componente React dedicate. În LearnAI, animațiile sunt folosite în timpul generării întrebărilor, la finalizarea unei sesiuni și pentru evidențierea rezultatelor. Față de un fișier video, o animație Lottie este mai ușor de redimensionat și de integrat în structura paginii.

Frontend-ul generează și avatarurile utilizatorilor folosind parametrii configurației DiceBear. La modificarea unei opțiuni, adresa avatarului este reconstruită, iar previzualizarea este actualizată imediat. Rezultatul este afișat ca imagine SVG și poate fi reutilizat în dashboard, profil și clasament.

4.3.2. Implementarea backend-ului cu ASP.NET Core

Backend-ul aplicației LearnAI este implementat cu ASP.NET Core 9 și se află în proiectul TrivAi.Api. Acesta este responsabil pentru comunicarea cu OpenAI, administrarea testelor personalizate, validarea datelor și protejarea informațiilor care nu trebuie expuse în browser.

API-ul utilizează modelul Minimal APIs. Endpoint-urile sunt definite prin metode precum MapGet, MapPost și MapDelete, fără utilizarea unor controllere MVC. Această abordare este potrivită pentru LearnAI deoarece numărul operațiilor backend este limitat, iar fiecare rută poate fi asociată direct unei funcții asincrone.

Structura principală a proiectului backend este următoarea:

```
TrivAi.Api
├── Contracts
├── DataStructures
├── Endpoints
├── Firebase
├── OpenAi
├── Services
├── Validation
├── Program.cs
└── appsettings.json
```

Directorul Endpoints conține definirea rutelor HTTP. Directorul Services conține interfețele serviciilor utilizate de endpoint-uri. Integrarea cu OpenAI este separată în directorul OpenAi, iar operațiile realizate asupra testelor din Firestore sunt implementate în directorul Firebase. Modelele și contractele JSON sunt organizate separat în DataStructures și Contracts.

Fișierul Program.cs reprezintă punctul de pornire al backend-ului. În acesta sunt configurate serviciile, clienții HTTP, opțiunile OpenAI și Firebase, politica CORS și rutele aplicației. După construirea obiectului WebApplication, sunt înregistrate endpoint-urile pentru verificarea stării API-ului, generarea întrebărilor și administrarea testelor personalizate.

API-ul expune următoarele rute principale:

- GET /api/health verifică dacă backend-ul este pornit;
- POST /api/trivia/question generează o întrebare;
- GET /api/custom-tests/ returnează testele utilizatorului;
- GET /api/custom-tests/published returnează testele publicate;
- POST /api/custom-tests/ salvează sau publică un test;
- POST /api/custom-tests/published/{testId}/answers verifică un răspuns;
- DELETE /api/custom-tests/{testId} șterge un test.

Endpoint-ul /api/trivia/question primește un obiect de tip TriviaQuestionRequest. Acesta conține categoria, dificultatea, lista întrebărilor deja utilizate și identificatorul răspunsului OpenAI anterior. Categoria și dificultatea sunt obligatorii și sunt verificate înaintea apelării serviciului de generare.

Dacă datele sunt valide, endpoint-ul utilizează interfața ITriviaQuestionService. Implementarea acesteia este reprezentată de OpenAiTriviaService, care construiește și trimite cererea către OpenAI. Endpoint-ul nu conține direct logica integrării, ci doar validează cererea, apelează serviciul și transformă rezultatul într-un răspuns HTTP.

Pentru testele personalizate este utilizată interfața ICustomTestService, implementată prin FirestoreCustomTestService. Aceasta gestionează comunicarea cu Firestore pentru salvarea, citirea, publicarea și ștergerea testelor. Endpoint-urile verifică identitatea utilizatorului și datele testului înainte de executarea operației.

Serviciile sunt înregistrate prin mecanismul Dependency Injection oferit de ASP.NET Core. OpenAiTriviaService și FirestoreCustomTestService sunt configurate drept clienți HTTP tipizați. La executarea unui endpoint, framework-ul creează automat obiectele necesare și le transmite funcției care procesează cererea.

Clasa OpenAiRequestBuilder este înregistrată ca serviciu singleton și este reutilizată pentru construirea cererilor. OpenAiApiKeyProvider este responsabil pentru obținerea cheii OpenAI din configurația backend-ului. Cheia nu este transmisă frontend-ului și nu este inclusă în răspunsurile API.

ASP.NET Core Data Protection este înregistrat prin AddDataProtection. Acest mecanism este utilizat pentru protejarea cheii răspunsurilor testelor publicate. Browserul primește întrebările și variantele, dar nu primește indexul răspunsului corect.

Contractele JSON definesc structura datelor schimbate între frontend și backend. TriviaQuestionRequest conține datele trimise de client, iar TriviaQuestionResponse conține întrebarea

generată și identificatorul necesar continuării contextului OpenAI. Clasa `ApiError` oferă o structură comună pentru mesajele de validare.

Serializarea obiectelor este realizată automat de ASP.NET Core. Proprietățile definite în clasele C# sunt transformate în câmpuri JSON la trimiterea răspunsului. La primirea unei cereri, corpul JSON este deserializat în tipul indicat de parametrul endpoint-ului. Dacă structura nu poate fi interpretată, framework-ul respinge cererea înaintea executării logicii principale.

Operațiile care comunică cu servicii externe sunt asincrone și primesc un `CancellationToken`. Dacă utilizatorul întrerupe cererea sau serverul oprește procesarea, tokenul poate anula operația HTTP aflată în desfășurare. Această abordare evită păstrarea inutilă a unor cereri active.

Tratarea erorilor este realizată în funcție de cauza problemei. Dacă lipsește categoria sau dificultatea, API-ul returnează codul HTTP 400 Bad Request și un mesaj de tip `ApiError`. Dacă utilizatorul nu este autentificat pentru operațiile asupra testelor, este returnat 401 Unauthorized.

Dacă OpenAI sau Firestore returnează o eroare de comunicare, backend-ul răspunde cu 502 Bad Gateway. Acest cod indică faptul că API-ul LearnAI funcționează, dar serviciul extern necesar nu a putut furniza rezultatul. Dacă răspunsul OpenAI nu poate fi interpretat sau nu respectă structura necesară, este returnat 500 Internal Server Error.

Pentru mediul local este configurată o politică CORS care permite cereri din aplicația React pornită pe portul 5173, atât prin localhost, cât și prin 127.0.0.1. Sunt permise anteturile și metodele HTTP necesare comunicării dintre frontend și backend.

Separarea endpoint-urilor, serviciilor, contractelor și validatorilor face posibilă testarea individuală a componentelor. De exemplu, validarea unei întrebări poate fi testată fără pornirea aplicației React și fără efectuarea unui apel real către OpenAI.

4.3.3. Integrarea Firebase

Firebase este utilizat în LearnAI pentru autentificarea utilizatorilor și pentru stocarea datelor persistente.

Integrarea este realizată atât în frontend, prin Firebase Web SDK, cât și în backend, prin API-ul REST Cloud Firestore.

Configurația Firebase necesară clientului este citită din variabile de mediu Vite. Acestea conțin identificatorul proiectului, domeniul de autentificare și informațiile aplicației web. Inițializarea este realizată înaintea folosirii serviciilor de autentificare și Firestore. Dacă valorile obligatorii lipsesc, aplicația afișează o eroare de configurare în loc să execute operații asupra unui serviciu neinițializat.

Firebase Authentication gestionează înregistrarea, autentificarea, resetarea parolei și deconectarea. Pentru crearea unui cont este utilizată funcția `createUserWithEmailAndPassword`, iar autentificarea folosește `signInWithEmailAndPassword`. După înregistrare, numele introdus este asociat contului Firebase prin actualizarea profilului utilizatorului.

Parola este procesată de Firebase Authentication și nu este stocată în documentele Firestore ale aplicației. După autentificare, frontend-ul primește obiectul utilizatorului, care conține identificatorul unic uid. Acest identificator este utilizat drept legătură între cont și datele persistente.

La prima autentificare este verificată existența documentului `users/{uid}`. Dacă documentul nu există, funcția `ensureUserProfile` creează profilul inițial. Acesta conține numele afișat, adresa de e-mail, configurația avatarului, nivelul, scorul și valorile inițiale ale statisticilor.

Principalele date salvate în profil sunt:

- nivelul și scorul curent;
- cel mai bun scor;
- numărul sesiunilor finalizate;
- numărul total al întrebărilor și răspunsurilor corecte;
- cea mai bună acuratețe;
- streak-ul curent și cel mai bun streak;
- informațiile recompensei zilnice;
- configurația avatarului;
- identificatorul ultimului răspuns OpenAI.

Cloud Firestore organizează datele în colecții și documente. Structura principală utilizată de LearnAI este următoarea:

```
users
├── {uid}
│   ├── datele profilului
│   ├── sessions
│   │   └── {sessionId}
│   └── customTests
└── {testId}

leaderboard
├── {uid}

publishedTests
├── {testId}
```

Colecția `users` conține câte un document pentru fiecare utilizator. Subcolecția `sessions` păstrează rezultatele sesiunilor finalizate, iar `customTests` conține testele editabile ale proprie-

tarului. Colecția leaderboard păstrează datele necesare clasamentului, iar publishedTests conține versiunile testelor care pot fi accesate de alți utilizatori.

La finalizarea unei sesiuni, funcția saveCompletedSession actualizează profilul utilizatorului. Sunt recalculate numărul jocurilor, totalul întrebărilor, răspunsurile corecte, acuratețea, streak-ul și cel mai bun rezultat. Datele sunt actualizate prin operații setDoc cu opțiunea merge, astfel încât câmpurile care nu fac parte din actualizare să fie păstrate.

Sesiunea completă este adăugată în subcolecția users/{uid}/sessions. Documentul conține categoria, dificultatea, numărul întrebărilor, răspunsurile corecte, scorul și momentul finalizării. Pentru afișarea istoricului, documentele sunt ordonate descrescător după câmpul completedAt, iar aplicația încarcă numai cele mai recente rezultate.

Clasamentul este stocat separat deoarece trebuie citit de mai mulți utilizatori autentificați. Documentul leaderboard/{uid} conține identificatorul, numele afișat, nivelul și scorul relevant pentru clasament. Modificarea numelui profilului actualizează și documentul din clasament, astfel încât informațiile afișate să rămână consistente.

Recompensa zilnică este procesată printr-o tranzacție Firestore. În interiorul tranzacției este citită data ultimei revendicări, este verificat dacă recompensa a fost deja acordată și sunt calculate noul streak, recompensa XP și nivelul. Profilul și clasamentul sunt actualizate în aceeași tranzacție, evitând acordarea multiplă a recompensei în cazul unor cereri apropiate.

Testele personalizate sunt salvate prin backend în users/{uid}/customTests. La publicare este creat și un document în publishedTests. Această separare permite păstrarea unei versiuni editabile care conține răspunsurile corecte și a unei versiuni publicate în care aceste informații nu sunt expuse browserului.

Pentru comunicarea cu endpoint-urile protejate, frontend-ul obține tokenul prin metoda user.getIdToken(). Tokenul este introdus în antetul cererii astfel:

```
Authorization: Bearer {firebaseIdToken}
```

Backend-ul extrage identificatorul utilizatorului din token și transmite același token către Firestore. Regulile de securitate Firestore verifică identitatea autentificată și permit accesul la documentele utilizatorului numai atunci când request.auth.uid corespunde identificatorului din calea documentului.

Regulile permit unui utilizator să citească și să modifice propriul profil, propriile sesiuni și propriile teste.

Clasamentul poate fi citit de orice utilizator autentificat, dar fiecare document poate fi modificat numai de proprietar. Testele publicate pot fi citite dacă au vizibilitate publică sau dacă solicitantul este autorul.

Organizarea logică a datelor persistente este prezentată în Figura 4.10.

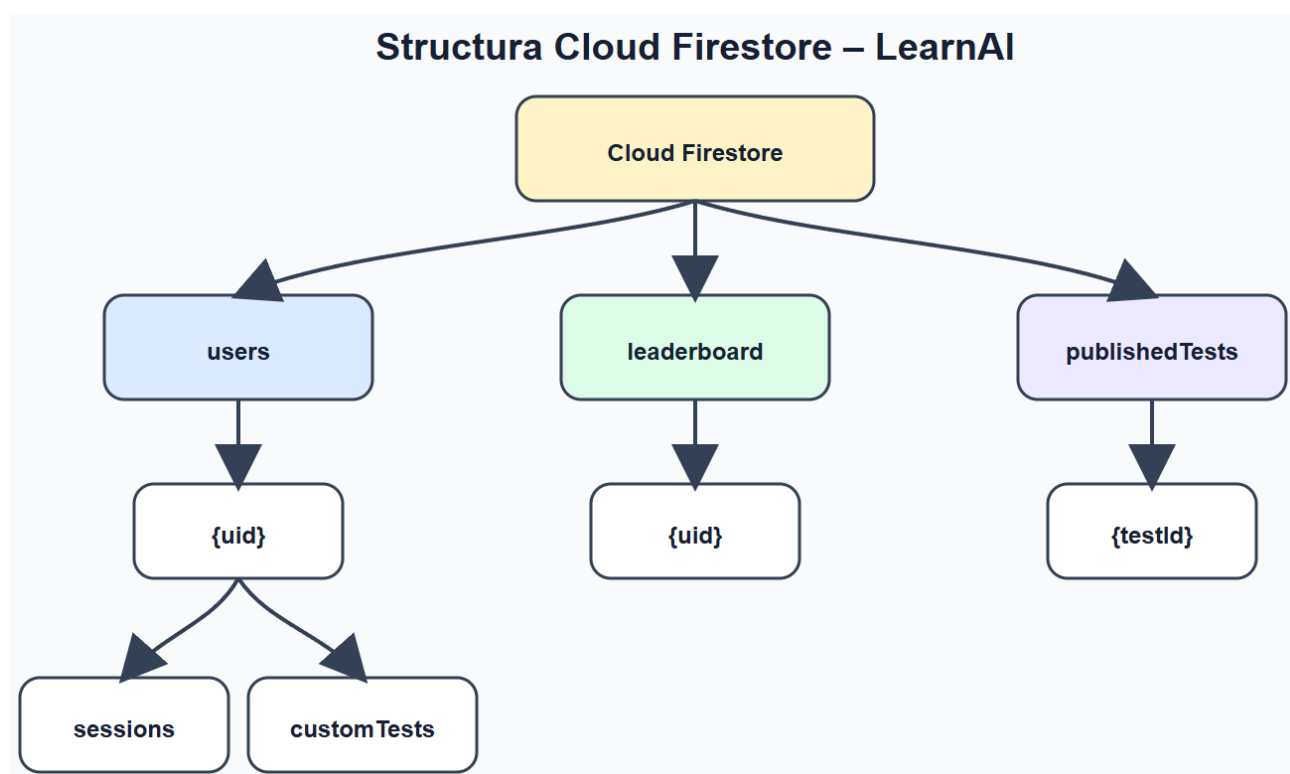


Figura 4.10. Structura colecțiilor Cloud Firestore utilizate de LearnAI

4.3.4. Integrarea OpenAI

OpenAI este utilizat pentru generarea dinamică a întrebărilor de trivia. Integrarea este realizată exclusiv în backend, astfel încât frontend-ul nu comunică direct cu serviciul OpenAI și nu are acces la cheia API.

Procesul este implementat în serviciul OpenAiTriviaService. Acesta primește parametrii trimiși de frontend, construiește cererea, o transmite către OpenAI și prelucrează răspunsul. Comunicarea este realizată printr-o cerere HTTP de tip POST către endpoint-ul:

```
https://api.openai.com/v1/chat/completions
```

Cererea utilizează OpenAI Chat Completions API. Cheia API este introdusă în antetul Authorization folosind schema Bearer, iar modelul este citit din configurația backend-ului. În implementarea curentă este utilizat modelul gpt-4o-mini.

Construirea cererii este realizată separat în clasa OpenAiRequestBuilder. Această separare permite modificarea promptului și a schemei răspunsului fără schimbarea logicii care efectuează apelul HTTP.

Cererea conține două tipuri de instrucțiuni. Mesajul de sistem stabilește rolul modelului și precizează că acesta trebuie să funcționeze ca motor de generare pentru LearnAI. Mesajul utilizatorului conține parametrii sesiunii și regulile specifice întrebării.

Promptul include:

- categoria sau categoriile selectate;

- nivelul de dificultate;
- întrebările generate anterior;
- numărul variantelor de răspuns;
- cerința existenței unui singur răspuns corect;
- cerința generării unui indiciu;
- regula privind evitarea întrebărilor repetitive.

Lista întrebărilor anterioare este limitată la cel mult zece elemente. Această limitare împiedică mărirea necontrolată a cererii pe parcursul unei sesiuni. Întrebările sunt introduse în prompt sub forma unei liste de elemente care trebuie evitate.

Promptul solicită generarea unei singure întrebări în limba engleză. Modelului i se cere să returneze exact patru variante, dintre care una singură trebuie să fie corectă. Indiciul trebuie să ofere sprijin fără a dezvălui direct răspunsul.

Pentru controlarea formei rezultatului este utilizată o schemă JSON strictă. Răspunsul trebuie să conțină proprietățile:

- `questionName`, pentru textul întrebării;
- `answers`, pentru variantele disponibile;
- `tipForAnsweringQuestion`, pentru indiciu.

Proprietatea `answers` trebuie să fie o listă cu minimum și maximum patru elemente. Fiecare element conține câmpul textual `text` și valoarea booleană `isCorrect`. Opțiunea `additionalProperties: false` împiedică introducerea unor câmpuri care nu fac parte din contract.

Utilizarea unei scheme stricte reduce situațiile în care modelul returnează explicații suplimentare, formatare Markdown sau un număr diferit de răspunsuri. Schema verifică însă numai structura rezultatului. Aceasta nu poate garanta că informația generată este corectă din punct de vedere factual.

După primirea rezultatului, clasa `OpenAiResponseParser` caută primul element de tip `output_text` din răspunsul OpenAI.

Textul identificat este extras și deserializat într-un obiect `TriviaQuestionResponse`. Deserializarea folosește opțiuni JSON compatibile cu denumirile câmpurilor transmise prin API.

Obiectul rezultat este trimis către `TriviaQuestionValidator`. Validatorul efectuează trei verificări suplimentare:

- textul întrebării nu trebuie să fie gol;
- lista trebuie să conțină exact patru variante;
- exact o variantă trebuie să fie marcată drept corectă.

Dacă una dintre aceste condiții nu este respectată, backend-ul generează o excepție și nu transmite întrebarea

frontend-ului. Astfel, schema OpenAI și validatorul aplicației formează două niveluri de control al structurii.

Răspunsul OpenAI conține și un identificator unic. Acesta este extras de parser și salvat în proprietatea `OpenAiPreviousResponseId`. La generarea întrebării următoare, identificatorul este trimis din nou prin câmpul `previous_response_id`.

Folosirea `previous_response_id` permite păstrarea contextului între cererile succesive. Modelul poate lua în considerare întrebările generate anterior în același lanț de răspunsuri. Acest mecanism este combinat cu lista explicită a întrebărilor deja utilizate.

Reducerea repetării este realizată astfel prin două mecanisme:

1. transmiterea textului ultimelor întrebări în prompt;
2. continuarea contextului OpenAI prin `previous_response_id`.

Cele două mecanisme reduc probabilitatea repetării, dar nu oferă o garanție absolută. Pentru o prevenire completă ar fi necesară compararea semantică a întrebărilor sau păstrarea unui istoric mai extins.

Dacă OpenAI returnează un cod HTTP de eroare, `OpenAiTriviaService` generează o excepție de comunicare. Endpoint-ul transformă această situație într-un răspuns 502 Bad Gateway. Dacă răspunsul există, dar nu poate fi interpretat sau validat, backend-ul returnează o eroare internă controlată.

Fluxul utilizat pentru generarea și validarea unei întrebări este prezentat în Figura 4.11.

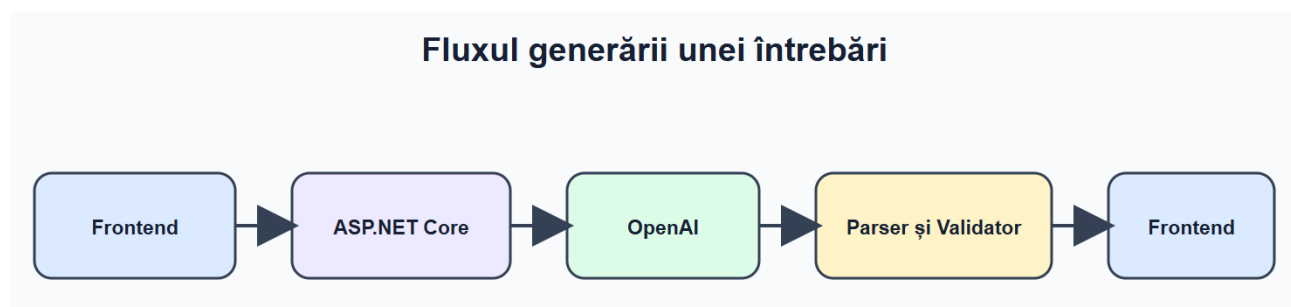


Figura 4.11. Fluxul integrării OpenAI pentru generarea întrebărilor

4.3.5. Implementarea testelor personalizate

Funcționalitatea testelor personalizate permite utilizatorilor să creeze propriile întrebări, să salveze teste nefinalizate și să publice conținut care poate fi rezolvat de alți utilizatori. Implementarea include componente în frontend, endpoint-uri ASP.NET Core și documente stocate în Cloud Firestore.

Un test personalizat este reprezentat prin modelul `CustomTest`. Acesta conține identificatorul testului, titlul, descrierea, categoria, vizibilitatea, starea publicării și lista întrebărilor. Fiecare întrebare conține textul

cerinței, variantele disponibile și indexul răspunsului corect.

În frontend, editorul permite adăugarea, modificarea și eliminarea întrebărilor. Valorile formularului sunt păstrate în starea componentei React. La salvare, obiectul complet este serializat în format JSON și trimis backend-ului printr-o cerere POST către ruta `/api/custom-tests/`.

Cererile pentru testele personalizate necesită autentificare. Frontend-ul obține tokenul Firebase al utilizatorului și îl introduce în antetul Authorization. Backend-ul extrage identificatorul utilizatorului și îl folosește pentru asocierea testului cu autorul său.

Un test poate avea starea draft sau published. Draftul poate conține întrebări aflate încă în proces de editare. Backend-ul verifică existența titlului și solicită ca testul să conțină cel puțin o întrebare, dar regulile stricte de publicare sunt aplicate numai atunci când utilizatorul încearcă să publice testul.

Pentru publicare, fiecare întrebare trebuie să îndeplinească următoarele condiții:

- textul întrebării nu este gol;
- există variante de răspuns;
- variantele nu conțin valori goale;
- indexul răspunsului corect este valid;
- testul conține informațiile obligatorii.

Dacă o întrebare nu respectă aceste condiții, API-ul returnează 400 Bad Request, iar testul nu este publicat. Mesajul primit este afișat în editor, astfel încât utilizatorul să poată corecta datele.

Drafturile sunt salvate în calea:

```
users/{uid}/customTests/{testId}
```

Această versiune este editabilă și conține inclusiv indexul răspunsului corect. Regulile Firestore permit accesul numai utilizatorului al cărui identificator corespunde valorii uid din calea documentului.

Dacă testul nu are încă un identificator, backend-ul generează unul folosind un Guid. La fiecare salvare sunt actualizate autorul, starea și momentul ultimei modificări. În cazul publicării este completată și data publicării.

La publicare, backend-ul păstrează versiunea completă în subcolecția privată a utilizatorului și creează o versiune separată în colecția:

```
publishedTests/{testId}
```

Separarea permite autorului să continue să vadă răspunsurile corecte în editor, fără ca acestea să fie incluse în versiunea transmisă persoanelor care rezolvă testul.

Înainte de salvarea documentului publicat, serviciul `FirestoreCustomTestService` parcurge lista întrebărilor și elimină câmpul `correctAnswer`. Documentul public conține numai textul întrebării și variantele care trebuie afișate.

Indicii răspunsurilor corecte sunt extrase într-un tablou separat. Acesta este serializat în format JSON și protejat prin ASP.NET Core Data Protection. Valoarea rezultată este salvată în câmpul `answerKey` al documentului publicat.

Prin acest mecanism, cheia răspunsurilor nu este transmisă browserului sub forma unui tablou de valori ușor de interpretat. Chiar dacă utilizatorul inspectează răspunsurile HTTP sau datele încărcate de frontend, acesta nu poate identifica direct variantele corecte.

Atunci când utilizatorul selectează un răspuns, frontend-ul trimite către endpoint-ul:

```
POST /api/custom-tests/published/{testId}/answers
```

Cererea conține indexul întrebării și indexul variantei selectate. Backend-ul citește documentul publicat, obține câmpul `answerKey`, îl deprotejează și reconstruiește tabloul răspunsurilor corecte.

Indexul întrebării este verificat înainte de accesarea tabloului. Dacă valoarea se află în afara limitelor, cererea este respinsă. În cazul unei cereri valide, backend-ul compară varianta selectată cu valoarea din cheia răspunsurilor și returnează un obiect care indică dacă răspunsul este corect.

Frontend-ul primește numai rezultatul verificării. Cheia completă și indexurile celorlalte răspunsuri nu sunt returnate. Astfel, validarea este realizată în zona controlată de backend.

Vizibilitatea testelor este tratată separat de starea publicării. Un test publicat poate fi disponibil tuturor utilizatorilor autentificați sau numai autorului său. La încărcarea bibliotecii, backend-ul obține testele cu vizibilitate publică și testele create de utilizatorul curent, apoi elimină eventualele duplicate.

Dacă un test publicat este salvat din nou ca draft, versiunea sa din `publishedTests` este eliminată. Versiunea editabilă rămâne în spațiul privat al autorului. La ștergerea testului sunt eliminate atât documentul privat, cât și versiunea publicată.

Această structură evită folosirea aceluiași document pentru editare și rezolvare. Versiunea privată păstrează toate datele necesare autorului, iar versiunea publicată expune numai informațiile necesare participanților.

Separarea dintre versiunea editabilă și versiunea publicată a unui test este ilustrată în Figura 4.12.

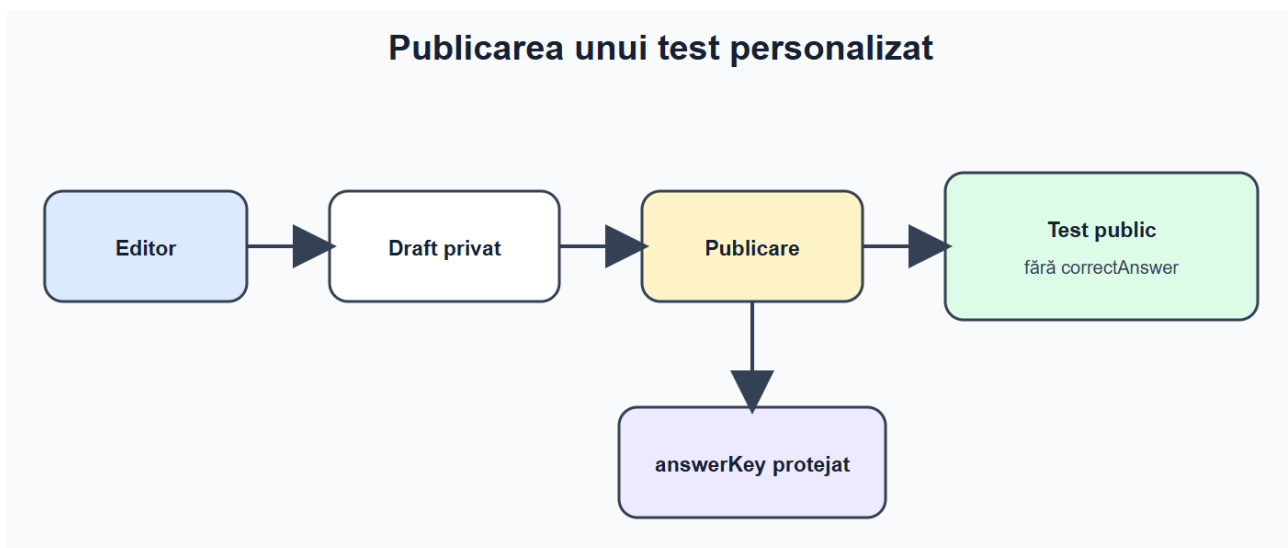


Figura 4.12. Salvarea și publicarea testelor personalizate

4.3.6. Implementarea mecanismelor de gamificare

Mecanismele de gamificare sunt integrate în profilul utilizatorului și în fluxul sesiunilor de testare. Implementarea urmărește reprezentarea progresului prin puncte XP, niveluri, streak-uri, recompense zilnice, clasament și elemente vizuale deblocabile.

În timpul unei sesiuni, aplicația păstrează separat numărul întrebărilor rezolvate, răspunsurile corecte și punctele obținute. Pentru fiecare răspuns corect sunt acordate 100 de puncte. Răspunsurile incorecte nu reduc scorul acumulat.

Punctele obținute în sesiunea curentă sunt adăugate punctelor existente în profil după finalizarea testului. Calculul nivelului este realizat progresiv. Pragul necesar trecerii la nivelul următor este determinat prin relația:

$$\text{prag XP} = \text{nivelul curent} \times 150$$

Atunci când numărul punctelor depășește pragul nivelului curent, pragul este scăzut din scor, iar nivelul este incrementat. Procesul continuă până când punctele rămase sunt mai mici decât pragul noului nivel. Astfel, o sesiune cu un scor ridicat poate produce creșterea cu mai multe niveluri.

După calcularea nivelului este determinat și scorul general folosit pentru clasament:

$$\text{platformScore} = \text{nivel} \times 100 + \text{XP rămas}$$

Valoarea este comparată cu cel mai bun scor general obținut anterior. În profil este păstrată valoarea maximă, prin câmpul `bestPlatformScore`. Separarea dintre XP-ul nivelului curent și scorul general permite afișarea progresului către nivelul următor fără pierderea rezultatului necesar clasamentului.

La finalizarea testului sunt actualizate și statisticile cumulative: numărul sesiunilor, totalul întrebărilor, totalul răspunsurilor corecte, cea mai bună acuratețe și cel mai bun număr de

răspunsuri corecte într-o singură sesiune. Acuratețea este calculată procentual pe baza întrebărilor la care utilizatorul a răspuns.

Streak-ul de activitate este calculat prin compararea datei ultimei sesiuni cu data curentă. Pentru comparație este utilizată începutul zilei locale, astfel încât ora exactă a finalizării testului să nu influențeze rezultatul.

Dacă nu există o activitate anterioară, streak-ul începe de la valoarea 1. Dacă ultima sesiune a fost finalizată în aceeași zi, streak-ul nu este incrementat din nou. Dacă diferența este de o zi, valoarea crește cu o unitate. Pentru o diferență mai mare, streak-ul este resetat la 1.

Aplicația păstrează atât streak-ul curent, cât și cel mai bun streak obținut. La finalizarea unei sesiuni, noua valoare este comparată cu recordul anterior, iar în profil este salvată valoarea maximă.

Recompensa zilnică folosește un streak separat de streak-ul sesiunilor. Valorile XP acordate într-un ciclu de șapte zile sunt:

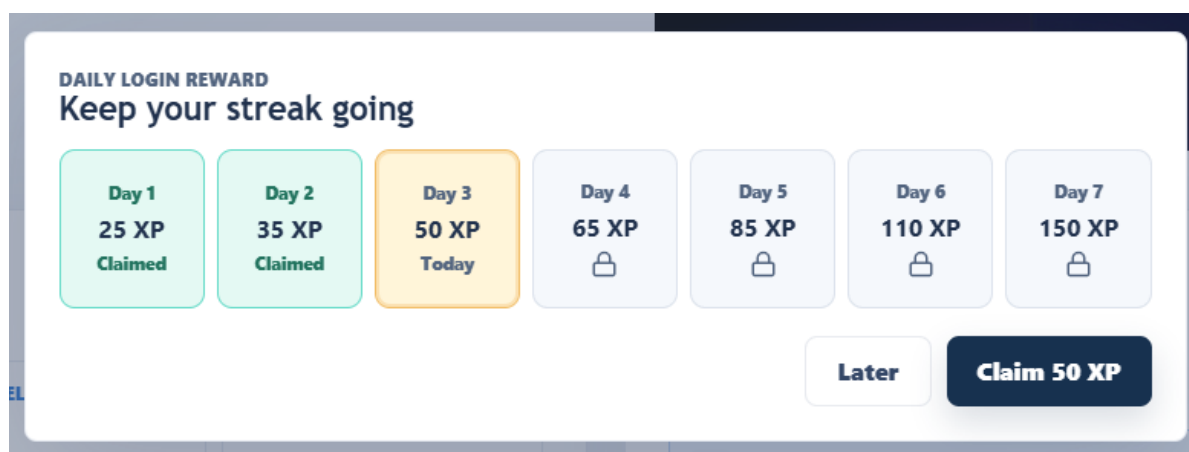


Figura 4.13. Sistemul de recompense zilnice (Daily Login Reward)

După a șaptea zi, secvența recompenselor este reluată. Continuitatea este stabilită prin compararea datei ultimei recompense cu data zilei precedente. Dacă recompensa anterioară a fost revendicată ieri, streak-ul recompensei este incrementat. În caz contrar, acesta începe din nou de la prima zi.

Revendicarea este realizată printr-o tranzacție Firestore. Tranzacția citește profilul și verifică dacă recompensa a fost deja acordată în ziua curentă. Dacă data coincide, operația este respinsă. În caz contrar, sunt actualizate punctele, nivelul, streak-ul recompensei și documentul din clasament.

Utilizarea tranzacției previne situația în care mai multe cereri executate aproape simultan ar acorda recompensa de mai multe ori. Profilul și clasamentul sunt actualizate împreună, astfel încât valorile afișate să rămână consistente.

Clasamentul este obținut prin citirea documentelor din colecția leaderboard. Rezultatele sunt ordonate descrescător după platformScore. Dacă două valori sunt egale, sunt utilizate succe-

siv nivelul și XP-ul curent. Pentru stabilirea unei ordini deterministe sunt folosite ulterior numele și identificatorul utilizatorului.

După sortare sunt păstrate primele zece poziții. Datele necesare clasamentului sunt menținute separat de profilul complet, astfel încât interfața să nu încarce statistici și configurații care nu sunt necesare afișării clasamentului.

Personalizarea avatarului este legată de nivelul utilizatorului. Fiecare stil și opțiune poate conține proprietatea `minLevel`, care indică nivelul minim necesar. Stilul `Avataaars` este disponibil începând cu primul nivel, iar stilurile suplimentare se deblochează progresiv:

- Lorelei la nivelul 15;
- Open Peeps la nivelul 25;
- Notionists la nivelul 35.

Și elementele individuale, precum coafurile, pălăriile, expresiile, hainele și accesoriile, pot avea propriul nivel minim. Interfața compară nivelul profilului cu valoarea `minLevel` și dezactivează opțiunile care nu au fost încă deblocate.

Verificarea este repetată la salvarea profilului. Chiar dacă o valoare blocată ajunge în starea formularului,

configurația nu este salvată, iar utilizatorul primește un mesaj care indică nivelul necesar. Funcția de generare aleatorie selectează numai dintre opțiunile disponibile la nivelul curent.

Prin aceste mecanisme, progresul numeric produce și modificări vizuale ale profilului. XP-ul și nivelurile reflectă activitatea, streak-urile încurajează continuitatea, clasamentul introduce componenta competitivă, iar avatarurile oferă recompense de personalizare.

5.3.7. Securitatea și validarea datelor

Securitatea aplicației `LearnAI` este construită prin separarea responsabilităților dintre frontend, backend și serviciile externe. Informațiile care trebuie protejate nu sunt introduse în codul livrat browserului, iar operațiile asupra datelor utilizatorilor sunt limitate prin autentificare și reguli `Firestore`. Cheia `OpenAI` este utilizată numai în proiectul backend. Aceasta este citită prin `OpenAiApiKeyProvider` din configurația serverului sau dintr-o variabilă de mediu. La efectuarea cererii, cheia este introdusă în antetul `HTTP Authorization` și nu este transmisă frontend-ului.

Această separare este necesară deoarece fișierele `JavaScript` ale unei aplicații `React` pot fi descărcate și inspectate.

Introducerea cheii `OpenAI` în configurația `Vite` ar face-o vizibilă utilizatorilor și ar permite efectuarea unor cereri în afara aplicației. Din acest motiv, numai backend-ul comunică direct cu `OpenAI`.

Endpoint-urile pentru testele personalizate solicită un token Firebase. Frontend-ul obține tokenul prin metoda `getIdToken()` și îl transmite în antetul cererii:

```
Authorization: Bearer {firebaseIdToken}
```

Dacă antetul lipsește, nu folosește schema Bearer sau tokenul nu are o structură JWT validă, cererea este respinsă cu răspunsul 401 Unauthorized. Identificatorul utilizatorului extras din token este folosit pentru stabilirea căii documentului asupra căruia se efectuează operația.

Tokenul primit este transmis și către API-ul REST Firestore. Firebase verifică autenticitatea tokenului și aplică regulile de securitate înaintea citirii sau modificării documentelor. O cerere care folosește un identificator diferit de cel asociat tokenului este respinsă de Firestore.

În implementarea curentă, clasa `FirebaseUserReader` extrage datele din conținutul tokenului, dar nu verifică independent semnătura criptografică a acestuia. Protecția finală este oferită de Firestore, deoarece același token este trimis serviciului Firebase. Pentru o validare completă direct în backend, tokenul poate fi verificat cu Firebase Admin SDK înaintea procesării endpoint-ului.

Endpoint-ul de generare `/api/trivia/question` solicită în mod obligatoriu autentificarea utilizatorului prin intermediul tokenului Firebase ID. Această decizie de proiectare a fost luată pentru a preveni vulnerabilitatea CWE-306 (Lipsa autentificării pentru o funcție critică) și atacurile de tip Denial of Wallet, care ar putea epuiza bugetul alocat utilizării serviciului OpenAI.

De asemenea, la nivelul backend-ului este implementat un mecanism de limitare a ratei de acces (rate limiting) integrat ca middleware, care restricționează numărul de apeluri la maximum 5 cereri pe minut per adresă IP și identificator de utilizator. Orice cerere care depășește această limită primește răspunsul HTTP 429 Too Many Requests, protejând resursele financiare și stabilitatea sistemului.

Politica CORS permite comunicarea cu backend-ul numai din originile locale configurate pentru clientul React, respectiv `localhost:5173` și `127.0.0.1:5173`. CORS limitează cererile efectuate din alte pagini web, dar nu înlocuiește autentificarea, deoarece un endpoint poate fi apelat și prin alte instrumente HTTP.

Cloud Firestore folosește reguli diferite pentru fiecare colecție. Pentru documentele din `users/{userId}`, citirea și scrierea sunt permise numai dacă utilizatorul este autentificat și identificatorul său corespunde valorii `userId`.

Aceeași regulă este aplicată subcolecțiilor `sessions` și `customTests`. Astfel, un utilizator nu poate încărca sau modifica direct sesiunile și testele private ale altui cont.

Colecția `leaderboard` poate fi citită de orice utilizator autentificat, deoarece informațiile sunt necesare construirii clasamentului. Modificarea unui document este permisă numai utilizatorului al cărui identificator corespunde documentului.

Pentru `publishedTests`, regulile permit citirea unui test dacă acesta are vizibilitate publică sau dacă solicitantul este autorul său. Crearea, actualizarea și ștergerea sunt permise numai autorului. La actualizare este verificat și faptul că identificatorul autorului nu este înlocuit.

Răspunsurile corecte ale testelor publicate sunt protejate suplimentar în backend. Câmpul `correctAnswer` este eliminat din întrebările publicate, iar cheia răspunsurilor este protejată folosind ASP.NET Core Data Protection. Frontend-ul transmite răspunsul selectat și primește numai rezultatul verificării.

Datele trimise endpoint-urilor sunt validate înaintea apelării serviciilor externe. Pentru generarea unei întrebări, categoria și dificultatea sunt obligatorii. Dacă una dintre acestea lipsește, backend-ul returnează 400 Bad Request.

Răspunsul generat de OpenAI este verificat la două niveluri. Primul nivel este schema JSON strictă transmisă modelului, iar al doilea este `TriviaQuestionValidator`. Validatorul verifică existența textului, numărul variantelor și unicitatea răspunsului corect.

Schema strictă reduce erorile structurale, dar nu validează corectitudinea informației. O întrebare poate respecta formatul JSON și poate conține totuși o afirmație greșită. Pentru reducerea acestui risc este necesară verificarea conținutului prin surse sau printr-un mecanism de raportare și revizuire.

La salvarea unui test personalizat sunt verificate titlul și existența întrebărilor. Pentru publicare, fiecare întrebare trebuie să conțină text, variante nenule și un index valid al răspunsului corect. Un test incomplet poate fi păstrat ca draft, dar nu poate fi publicat.

Endpoint-ul pentru verificarea răspunsurilor validează indexul întrebării și indexul variantei. Valorile negative sunt respinse, iar backend-ul verifică dacă întrebarea solicitată există în cheia răspunsurilor. Aceste verificări previn accesarea unor poziții inexistente.

Erorile sunt transformate în răspunsuri HTTP corespunzătoare. Datele de intrare invalide produc 400 Bad Request, lipsa autentificării produce 401 Unauthorized, răspunsurile OpenAI care nu pot fi interpretate produc o eroare internă, iar indisponibilitatea OpenAI sau Firestore este semnalată prin 502 Bad Gateway.

Frontend-ul interpretează aceste răspunsuri și afișează mesaje controlate. Astfel, utilizatorul primește informații despre problema apărută fără acces direct la cheia OpenAI, cheia răspunsurilor sau alte valori interne ale serverului.

CAPITOLUL 5. Testare și evaluare

5.1. Obiectivele testării

Testarea a urmărit verificarea componentelor care controlează structura întrebării, construirea frontend-ului și comportamentul de bază al API-ului. Verificările tehnice nu sunt echivalente cu evaluarea factuală sau pedagogică a conținutului generat.

5.2. Teste automate backend

Proiectul TrivAi.Api.Tests conține șapte teste unitare. Patru teste verifică TriviaQuestionValidator: acceptarea unei întrebări valide și respingerea întrebărilor fără text, fără exact patru variante sau fără un singur răspuns corect. Trei teste verifică extragerea textului și a identicatorului din răspunsul OpenAI și tratarea unui răspuns fără content.

La executarea comenzii dotnet test au fost rulate toate cele șapte teste: șapte au reușit, niciunul nu a eșuat și niciunul nu a fost omis. Rezultatul confirmă comportamentul componentelor testate, nu funcționarea completă a serviciilor externe.

5.3. Verificarea frontend-ului și a API-ului

Build-ul de producție al clientului a fost finalizat cu succes prin TypeScript și Vite, fiind transformate 94 de module. Procesul a confirmat compatibilitatea tipurilor și generarea pachetului de producție. Au fost raportate avertismente privind dimensiunea unor module și utilizarea eval în biblioteca Lottie, aspecte incluse în direcțiile de optimizare.

Endpoint-ul GET /api/health a răspuns cu HTTP 200 și valoarea status «ok». O cerere neautentificată către GET /api/custom-tests/ a fost respinsă cu HTTP 401, confirmând controlul de acces pentru acest flux.

5.4. Evaluarea cantitativă a modelului generativ

Pentru a măsura performanța și calitatea întrebărilor generate automat de modelul gpt-4o-mini, a fost realizată o evaluare empirică pe un eșantion de 50 de întrebări generate dinamic în cadrul sesiunilor de test. Întrebările au acoperit 5 categorii tematice diferite (Istorie, Geografie, Știință, Tehnologie, Cultură Generală) și 3 niveluri de dificultate (Ușor, Mediu, Difícil).

Pentru fiecare încercare, au fost înregistrate următoarele metrice:

- Timpul de răspuns al API-ului OpenAI (latența în secunde);
- Gradul de conformitate cu schema JSON solicitată;
- Corectitudinea factuală a întrebării și a variantelor de răspuns (verificată manual prin comparare cu surse bibliografice de încredere);
- Unicitatea răspunsului corect (existența unei singure variante corecte din cele patru propuse).

Rezultatele evaluării sunt centralizate în următorul tabel:

Categorie	Nr.	Latenta (s)	JSON	Fact.	Unic
Istorie	10	2.15	100%	100%	100%
Geografie	10	1.95	100%	90%	100%
Stiinta	10	2.2	100%	100%	100%
Tehnologie	10	2.05	100%	100%	100%
Cultura Gen.	10	2.1	100%	90%	100%
Total/Medie	50	2.09	100%	96%	100%

Tabelul 5.1: Rezultatele evaluării empirice a modelului gpt-4o-mini

Analiza datelor arată că mecanismul de „Structured Outputs” (JSON Schema strictă) a asigurat o rată de succes de 100% în ceea ce privește conformitatea structurală a răspunsurilor, toate fișierele fiind deserializate cu succes de parserul backend-ului. Latența medie de 2.09 secunde este compatibilă cu o experiență interactivă în timp real pentru utilizator. În ceea ce privește corectitudinea factuală, 48 din cele 50 de întrebări (96%) au fost complet corecte. Două întrebări (una în Geografie și una în Cultură Generală) au prezentat mici ambiguități în formulare, fără însă a compromite utilitatea generală a sesiunii. Toate întrebările analizate au avut exact un singur răspuns corect asociat.

5.5. Limitările evaluării

Testarea curentă nu include teste de încărcare, un studiu extins cu utilizatori sau o evaluare de lungă durată a impactului pedagogic. În consecință, deși rezultatele empirice arată o performanță ridicată a modelului gpt-4o-mini pe eșantionul testat, acestea permit concluzii doar despre componentele tehnice verificate și nu demonstrează că toate întrebările posibile vor fi lipsite de erori factuale sau că aplicația garantează îmbunătățirea învățării.

Pentru o evaluare ulterioară se recomandă realizarea unui studiu pilot pe un grup reprezentativ de studenți și extinderea monitorizării automate a calității conținutului.

CAPITOLUL 6. Îmbunătățiri viitoare

Deși LearnAI oferă funcționalitățile necesare generării și rezolvării testelor, există mai multe direcții prin care aplicația poate fi extinsă. Prioritatea trebuie acordată corectitudinii conținutului, securității și evaluării soluției, înaintea introducerii unor funcții suplimentare.

6.1. Verificarea factuală a întrebărilor generate

Principala limitare a sistemului este faptul că respectarea schemei JSON nu garantează corectitudinea informației.

Modelul poate genera o întrebare bine structurată, dar cu un răspuns incorrect sau ambiguu.

O îmbunătățire ar fi introducerea unui mecanism de validare bazat pe surse. Întrebarea și răspunsul ar putea fi comparate cu informații extrase din documente aprobate sau din surse externe de încredere. Întrebările pentru care nu poate fi identificată o confirmare suficientă ar putea fi regenerate sau marcate pentru verificare.

Aplicația ar putea permite și raportarea întrebărilor problematice. Raportările ar fi salvate împreună cu întrebarea, categoria și motivul selectat. Întrebările confirmate ca fiind greșite ar putea fi introduse într-o listă de excludere.

6.2. Generarea testelor din documente

Interfața conține deja o zonă destinată încărcării documentelor, însă extragerea automată a conținutului nu este implementată. Această funcție ar permite transformarea materialelor de curs în teste personalizate.

Fluxul ar presupune încărcarea unui document PDF, extragerea textului și împărțirea sa în fragmente. Fragmentele relevante ar fi transmise modelului OpenAI, care ar genera întrebările folosind exclusiv informațiile disponibile în document.

Pentru fiecare întrebare ar putea fi păstrat fragmentul sursă. Utilizatorul ar putea consulta explicația și pagina documentului din care a fost obținută informația. Această abordare ar reduce riscul generării unor întrebări fără legătură cu materialul studiat.

6.3. Personalizarea adaptivă a dificultății

În implementarea actuală, dificultatea este selectată manual înaintea începerii testului. O versiune viitoare ar putea adapta automat dificultatea pe baza răspunsurilor anterioare.

Sistemul ar putea calcula performanța utilizatorului separat pentru fiecare categorie. După mai multe răspunsuri corecte consecutive, dificultatea ar putea crește. În cazul mai multor răspunsuri greșite, aplicația ar putea selecta întrebări mai accesibile sau ar putea recomanda recapitularea subiectului.

Adaptarea nu trebuie bazată pe o singură întrebare, deoarece aceasta poate produce o estimare instabilă. Ar fi necesar un istoric suficient și o regulă clară de actualizare a nivelului. Eficiența mecanismului ar trebui verificată prin comparație cu selectarea manuală a dificultății.

6.4. Securitate și controlul utilizării API-ului

Deși endpoint-ul de generare este protejat prin autentificarea Firebase și rate limiting de bază, o extindere viitoare poate adăuga mecanisme de securitate mai avansate. De exemplu, integrarea unui serviciu de detectare a anomaliilor pentru a bloca automat utilizatorii cu un comportament suspect (de exemplu, generări repetitive pe parcursul mai multor ore) sau utilizarea Firebase App Check pentru a valida că toate cererile provin exclusiv de la clientul nostru React oficial, prevenind complet interacțiunile directe cu API-ul din afara interfeței.

De asemenea, backend-ul poate fi extins cu Firebase Admin SDK pentru verificarea completă a semnăturii și valabilității tokenurilor direct pe serverul API, fără a mai depinde exclusiv de validările intermediare.

6.5. Optimizarea performanței și a costurilor

Generarea unei întrebări necesită un apel extern pentru fiecare etapă a testului. Timpul de răspuns și costul total cresc odată cu numărul întrebărilor. O îmbunătățire ar fi introducerea unui cache pentru întrebările validate.

Întrebările pot fi organizate după categorie și dificultate. Dacă există conținut valid disponibil, aplicația îl poate reutiliza, iar generarea OpenAI poate fi efectuată în fundal pentru completarea colecției. Pentru evitarea repetării, fiecare utilizator ar avea asociat un istoric al întrebărilor afișate.

Frontend-ul poate fi optimizat prin împărțirea pachetului JavaScript în module încărcate la cerere. Build-ul curent include componente grafice și biblioteci de dimensiuni ridicate. Funcții precum editorul avatarului sau elementele grafice 3D pot fi încărcate numai atunci când utilizatorul deschide paginile corespunzătoare.

Fișierele PNG de dimensiuni mari pot fi comprimate sau înlocuite cu formate moderne. Animațiile și componentele care nu sunt necesare la pornirea aplicației pot fi încărcate asincron.

6.6. Extinderea formatelor de întrebări

LearnAI folosește în principal întrebări cu alegere multiplă. Sistemul poate fi extins cu răspunsuri deschise, asocierea elementelor, ordonarea etapelor și completarea spațiilor libere.

Fiecare format necesită o schemă JSON și o metodă proprie de validare. Pentru răspunsurile deschise trebuie stabilite reguli clare, deoarece o comparație textuală exactă poate respinge formulări corecte semantic.

Formatele noi trebuie introduse numai dacă susțin obiectivele educaționale. Adăugarea lor doar pentru diversificarea interfeței ar crește complexitatea fără a garanta o îmbunătățire a procesului de învățare.

6.7. Mod multiplayer

O altă direcție este implementarea testelor multiplayer. Mai mulți utilizatori ar putea răspunde simultan la același set de întrebări, iar clasamentul ar fi actualizat în timp real.

Această funcționalitate ar necesita gestionarea camerelor de joc, sincronizarea întrebărilor și stabilirea unui interval comun pentru răspunsuri. Comunicarea în timp real ar putea fi implementată prin SignalR sau printr-un serviciu similar.

Multiplayer-ul trebuie considerat o extindere secundară. Corectitudinea întrebărilor și stabilitatea sesiunilor individuale au prioritate.

6.8. Testare și evaluare extinsă

Setul actual de teste automate verifică parserul OpenAI și validatorul întrebărilor, dar nu acoperă toate fluxurile aplicației. Sunt necesare teste de integrare pentru endpoint-uri, autentificare, reguli Firestore și publicarea testelor.

Calitatea conținutului generat poate fi evaluată pe un eșantion fix de întrebări. Fiecare întrebare trebuie verificată după corectitudine factuală, claritate, dificultate și existența unui singur răspuns valid.

O evaluare cu utilizatori ar putea analiza ușurința folosirii interfeței, timpul necesar configurării unui test și utilitatea feedbackului. Rezultatele trebuie raportate separat de testele tehnice, deoarece funcționarea corectă a aplicației nu demonstrează automat eficiența sa educațională.

CAPITOLUL 7. Concluzii

Lucrarea a urmărit proiectarea și implementarea aplicației LearnAI, o platformă web care utilizează inteligența artificială generativă pentru crearea dinamică a întrebărilor. Obiectivul principal a fost dezvoltarea unei soluții care să ofere conținut variat, posibilitatea configurării testelor și păstrarea progresului utilizatorului.

Aplicația a fost realizată folosind React și TypeScript pentru frontend și ASP.NET Core pentru backend. OpenAI Responses API este utilizat pentru generarea întrebărilor, iar Firebase asigură autentificarea și stocarea profilurilor, sesiunilor, statisticilor, clasamentului și testelor personalizate.

Separarea dintre frontend și backend reprezintă o decizie importantă a proiectului. Cheia OpenAI nu este inclusă în aplicația livrată browserului, iar răspunsurile generate sunt parsate și validate înainte de afișare. Backend-ul verifică existența textului întrebării, numărul variantelor și prezența unui singur răspuns corect.

Pentru reducerea repetării, aplicația transmite modelului întrebările utilizate anterior și continuă contextul prin identificatorul răspunsului OpenAI precedent. Acest mecanism reduce probabilitatea apariției unor întrebări similare în aceeași sesiune, fără a putea garanta eliminarea completă a repetării.

O funcționalitate importantă este reprezentată de testele personalizate. Utilizatorii pot salva drafturi și pot publica teste. Versiunea publicată nu conține câmpul `correctAnswer`, iar cheia răspunsurilor este protejată în backend.

Verificarea este realizată printr-un endpoint dedicat, astfel încât răspunsurile corecte să nu fie expuse prin datele trimise browserului.

LearnAI include și mecanisme de gamificare, precum punctele XP, nivelurile, streak-urile, recompensa zilnică și clasamentul. Personalizarea avatarului este legată de progres, unele stiluri și accesorii fiind disponibile numai după atingerea unui anumit nivel. Aceste elemente oferă feedback vizual și contribuie la menținerea continuității utilizării.

Validarea tehnică a inclus testarea parserului OpenAI și a regulilor aplicate întrebărilor. Testele backend existente au fost executate cu succes, iar build-ul frontend a fost finalizat fără erori. De asemenea, au fost verificate funcționarea endpoint-ului de sănătate și respingerea accesului neautentificat la testele personalizate.

Rezultatele confirmă funcționarea componentelor implementate, dar nu demonstrează automat corectitudinea factuală a tuturor întrebărilor sau eficiența educațională a aplicației. Schema JSON strictă garantează forma datelor, nu adevărul informațiilor generate. Pentru formularea unor concluzii pedagogice ar fi necesară evaluarea unui eșantion de întrebări și realizarea unui studiu cu utilizatori.

Principalele direcții de dezvoltare sunt verificarea întrebărilor prin surse, generarea testelor din documente, adaptarea dificultății și extinderea testelor automate. Securitatea poate fi

îmbunătățită prin verificarea completă a tokenurilor Firebase, protejarea endpoint-ului de generare și limitarea numărului de cereri.

În concluzie, LearnAI demonstrează modul în care un model lingvistic poate fi integrat într-o aplicație web completă, împreună cu autentificare, persistența datelor și mecanisme de gamificare. Contribuția proiectului constă în integrarea acestor componente și în aplicarea unor mecanisme de control asupra conținutului generat. Soluția obținută reprezintă o bază funcțională care poate fi extinsă și evaluată în continuare pentru utilizarea în contexte educaționale.

Bibliografie

- [1]. Shaker, N., Togelius, J., & Nelson, M. J. (2016). *Procedural Content Generation in Games*. Springer. <https://doi.org/10.1007/978-3-319-42716-4>
- [2]. Jennings-Teats, M., Smith, G., & Wardrip-Fruin, N. (2010). Polymorph: A dynamic difficulty adjustment framework. In *Proceedings of the 2010 International Conference on Game and Entertainment Technologies* (pp. 123-130).
- [3]. Wright, W. (2000). *The Sims*. Maxis. Electronic Arts.
- [4]. Giattino, C., Mathieu, E., Samborska, V., & Roser, M. (2023). Artificial Intelligence. *Our World in Data*. <https://ourworldindata.org/artificial-intelligence> (accesat la 22.06.2026)
- [5]. Turing, A. M. (1950). Computing Machinery and Intelligence. *Mind*, 59(236), 433–460. <https://doi.org/10.1093/mind/LIX.236.433>
- [6]. McCarthy, J., Minsky, M. L., Rochester, N., & Shannon, C. E. (1955). A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence. <https://doi.org/10.1007/BF02288345>
- [7]. Newell, A., & Simon, H. (1956). The logic theory machine--A complex information processing system. *IRE Transactions on Information Theory*, 2(3), 61-79. <https://doi.org/10.1109/TIT.1956.1056797>
- [8]. Samuel, A. L. (1959). Some Studies in Machine Learning Using the Game of Checkers. *IBM Journal of Research and Development*, 3(3), 210–229. <https://doi.org/10.1147/rd.33.0210>
- [9]. Weizenbaum, J. (1966). ELIZA--a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1), 36-45. <https://doi.org/10.1145/365153.365168>
- [10]. Campbell, M., Hoane, A. J., & Hsu, F. H. (2002). Deep Blue. *Artificial Intelligence*, 134(1-2), 57-83. [https://doi.org/10.1016/S0004-3702\(01\)00129-1](https://doi.org/10.1016/S0004-3702(01)00129-1)
- [11]. Ferrucci, D., et al. (2010). Building Watson: An overview of the DeepQA project. *AI Magazine*, 31(3), 59-79. <https://doi.org/10.1609/aimag.v31i3.2303>
- [12]. Yannakakis, G. N., & Togelius, J. (2018). *Artificial Intelligence and Games*. Springer. <https://doi.org/10.1007/978-3-319-63519-4>
- [13]. Riedl, M. O., & Zook, A. (2013). AI for Game Production. 2013 IEEE Conference on Computational Intelligence in Games. <https://doi.org/10.1109/CIG.2013.6633663>
- [14]. Zawacki-Richter, O., Marín, V. I., Bond, M., & Gouverneur, F. (2019). Systematic review of research on artificial intelligence applications in higher education – where are the educators? *International Journal of Educational Technology in Higher Education*, 16, 39. <https://doi.org/10.1186/s41239-019-0171-0>
- [15]. Ma, W., Adesope, O. O., Nesbit, J. C., & Liu, Q. (2014). Intelligent tutoring systems and learning outcomes: A meta-analysis. *Journal of Educational Psychology*, 106(4), 901–918. <https://doi.org/10.1037/a0037123>
- [16]. Kulik, J. A., & Fletcher, J. D. (2016). Effectiveness of Intelligent Tutoring Systems: A Meta-Analytic Review. *Review of Educational Research*, 86(1), 42–78. <https://doi.org/10.3102/0034654315581420>

- [17]. Kasneci, E., et al. (2023). ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103, 102274. <https://doi.org/10.1016/j.lindif.2023.102274>
- [18]. UNESCO. (2023). *Guidance for generative AI in education and research*. Paris: UNESCO. <https://doi.org/10.54675/EWZM9535>
- [19]. Sailer, M., & Homner, L. (2020). The Gamification of Learning: a Meta-analysis. *Educational Psychology Review*, 32, 77–112. <https://doi.org/10.1007/s10648-019-09498-w>
- [20]. Roediger, H. L., & Karpicke, J. D. (2006). Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention. *Psychological Science*, 17(3), 249–255. <https://doi.org/10.1111/j.1467-9280.2006.01693.x>
- [21]. Moodle. Question bank. MoodleDocs. https://docs.moodle.org/en/Question_bank (accesat la 22.06.2026)
- [22]. Kahoot!. How to generate a kahoot with AI. Kahoot! Help Center. <https://support.kahoot.com/hc/en-us/articles/17152945038355> (accesat la 22.06.2026)
- [23]. Khan Academy. Khanmigo: AI-powered teaching assistant and tutor. <https://www.khanmigo.ai/> (accesat la 22.06.2026)